

Qudit-ADAPT para multiway number partitioning ($k = 3$)

Informe técnico

30 de septiembre de 2026

Resumen

Aplicamos Qudit-ADAPT al problema de repartir n números en tres subconjuntos de igual suma, con un qutrit por número. Hay dos bloques de simulaciones en BitWit, ambos con 20 instancias de solución única por tamaño, cada una bajo 5 órdenes de sus números:

- un estudio detallado con $n = 5-9$ (3378 simulaciones: $\ell = 1$ contra $\ell = 2$ y estrategias de inicialización, secciones 4.1–4.2);
- un benchmark ampliado de ADAPT ($k \leq 80$) contra QAOA ($p = 40$) con $n = 5-12$ (sección 4.4); INTERP de $n = 10-12$ está en curso.

Resultados principales:

- El problema es Max 3-Cut ponderado sobre el grafo completo K_n (pesos $a_i a_j$) más una constante. El pool es el de K_n : $2n^2 - n$ operadores con $\ell = 1$ y $3n + 40\binom{n}{2} + 78\binom{n}{3} + 32\binom{n}{4}$ con $\ell = 2$.
- En $n = 5-9$ el pool $\ell = 2$ resuelve más instancias que $\ell = 1$: gana en 40 instancias contra 10 (prueba de signo, $p = 2.4 \times 10^{-5}$). Con ambos pools la tasa de éxito cae con n . En $n = 10-12$, con tope de 80 parámetros, $\ell = 2$ queda limitado por el tope; aun así, en $n = 11$ y 12 resuelve algunas instancias y $\ell = 1$ casi ninguna.
- La conjetura de que el warm start es una desventaja no se sostiene. Con $\ell = 2$, warm resuelve más que ADAPT desde $\theta = 0$ (38 instancias contra 9, $p = 2.5 \times 10^{-5}$) y más que su propia secuencia reoptimizada desde $\theta = 0$ (32 contra 8, $p = 1.8 \times 10^{-4}$). Con $\ell = 1$ no hay diferencia significativa.
- Contra QAOA con 80 parámetros ($n = 5-12$), ADAPT da un $p_{\text{éxito}}$ medio mucho mayor que QAOA con reinicios aleatorios: 1750 veces en $n = 12$, donde QAOA queda al nivel del azar. Con inicialización INTERP, QAOA mejora mucho. ADAPT le gana en $p_{\text{éxito}}$ medio en todo n y en fracción resuelta desde $n = 8$, pero INTERP resuelve más instancias con $p_{\text{éxito}} \geq 0.1$ en $n \leq 7$.
- El resultado de una corrida individual depende de qué número se asigna a qué qutrit, una elección arbitraria. Desde la superposición uniforme hay operadores empatados en gradiente, y argmax toma el primero de la lista; cambiar la numeración cambia cuál es el primero y puede convertir un éxito en un fracaso (ejemplo en la sección 3.2). Por eso cada instancia se corre con 5 órdenes y se reporta la fracción que la resuelve.

Contenido

1	Problema y codificación	2
1.1	El problema	2
1.2	Costo: de la suma de cuadrados a Max 3-Cut	2
1.3	Codificación en qutrits	4
1.4	El pool de operadores	4
2	Implementación	8
2.1	El algoritmo	8
2.2	El circuito	9
2.3	Simulación clásica	9

3	Protocolo	10
3.1	Instancias	10
3.2	Órdenes	11
3.3	Ajustes y estrategias de inicialización	12
3.4	Análisis de datos	12
4	Resultados	13
4.1	$\ell = 1$ contra $\ell = 2$	13
4.2	Warm start contra $\theta = 0$	14
4.3	Comparación con QAOA	15
4.4	Benchmark ampliado: ADAPT ($k \leq 80$) contra QAOA ($p = 40$)	16
4.5	Compuertas nativas	18
4.6	Comparación al estilo del paper	19
4.7	Una métrica normalizada	20
4.8	¿Por qué ADAPT puede terminar peor que el azar?	21
4.9	Varios órdenes como reinicios	21
4.10	Varianza del gradiente: mapa en n y k	22
4.11	Sensibilidad al orden, a la máquina y al rango	22
4.12	Otras observaciones	23
5	Conclusiones	24
6	Notas para el manuscrito	25
7	Pendientes	26

1 Problema y codificación

1.1 El problema

Dados n enteros positivos $a_1, \dots, a_n$ con suma S , se busca asignar a cada uno una caja $z_j \in \{0, 1, 2\}$ de modo que las sumas por caja,

$$\Sigma_i(z) = \sum_{j=1}^n a_j \delta_{i,z_j}, \quad i = 0, 1, 2, \quad (1)$$

sean lo más parejas posible. Una partición es *perfecta* si $\Sigma_0 = \Sigma_1 = \Sigma_2 = S/3$.

Dos propiedades que usamos más adelante:

- **Degeneración mínima 6.** Permutar los nombres de las tres cajas no cambia las sumas, así que cada partición aparece como $3! = 6$ asignaciones z distintas. Una instancia tiene solución *única* si su fundamental tiene degeneración exactamente 6.
- **$n \geq 5$ con números distintos.** Una partición perfecta no puede dejar cajas vacías, porque $S/3 > 0$. Con $n = 3$ o $n = 4$ hay al menos dos cajas de un solo número, que tendrían que ser iguales. Con $n = 5$ la repartición $(2, 2, 1)$ sí es posible.

1.2 Costo: de la suma de cuadrados a Max 3-Cut

Seguimos el costo de J. Molina, con $\mu = S/3$:

$$C(z) = \sum_{i=0}^2 (\Sigma_i(z) - \mu)^2, \quad (2)$$

que vale 0 exactamente en una partición perfecta. Lo reescribimos en siete pasos.

Paso 1: las tres cajas suman S . Cada número está en exactamente una caja, así que $\sum_i \delta_{i,z_j} = 1$ para todo j . Entonces

$$\sum_i \Sigma_i = \sum_j a_j \sum_i \delta_{i,z_j} = \sum_j a_j = S. \quad (3)$$

Paso 2: desarrollar el cuadrado. Sumar μ^2 sobre las tres cajas da $3\mu^2$:

$$C = \sum_i \Sigma_i^2 - 2\mu \sum_i \Sigma_i + 3\mu^2 = \sum_i \Sigma_i^2 - 2\frac{S}{3}S + 3\frac{S^2}{9} = \sum_i \Sigma_i^2 - \frac{S^2}{3}. \quad (4)$$

Todo el problema queda en $\sum_i \Sigma_i^2$, la suma de los cuadrados de las sumas de las cajas.

Paso 3: expandir Σ_i^2 . Cada Σ_i es una suma sobre j , y su cuadrado es una doble suma:

$$\Sigma_i^2 = \left(\sum_j a_j \delta_{i,z_j} \right) \left(\sum_k a_k \delta_{i,z_k} \right) = \sum_{j,k} a_j a_k \delta_{i,z_j} \delta_{i,z_k}. \quad (5)$$

El producto $\delta_{i,z_j} \delta_{i,z_k}$ vale 1 sólo si j y k están ambos en la caja i . Al sumar sobre i sobrevive a lo más un término, el de la caja común, y sólo si comparten caja: $\sum_i \delta_{i,z_j} \delta_{i,z_k} = \delta_{z_j,z_k}$. Por lo tanto

$$\sum_i \Sigma_i^2 = \sum_{j,k} a_j a_k \delta_{z_j,z_k}. \quad (6)$$

Paso 4: separar $j = k$ de $j \neq k$. En la diagonal, $\delta_{z_j,z_j} = 1$ y queda $\sum_j a_j^2$. Fuera de la diagonal, cada par aparece dos veces, como (j, k) y como (k, j) :

$$\sum_i \Sigma_i^2 = \sum_j a_j^2 + 2 \sum_{j < k} a_j a_k \delta_{z_j,z_k}. \quad (7)$$

Paso 5: de “misma caja” a “cajas distintas”. Con $\delta = 1 - (1 - \delta)$, y con $w_{jk} = a_j a_k$,

$$\sum_{j < k} w_{jk} \delta_{z_j,z_k} = P - W_{\text{cut}}(z), \quad P = \sum_{j < k} w_{jk}, \quad W_{\text{cut}}(z) = \sum_{j < k} w_{jk} (1 - \delta_{z_j,z_k}). \quad (8)$$

$1 - \delta_{z_j,z_k}$ vale 1 justo cuando el par queda *cortado* (en cajas distintas). P es el peso de todos los pares, y W_{cut} el de los pares cortados.

Paso 6: P sólo depende de los números. De $S^2 = (\sum_j a_j)^2 = \sum_j a_j^2 + 2 \sum_{j < k} a_j a_k$,

$$P = \frac{1}{2} (S^2 - \sum_j a_j^2). \quad (9)$$

Paso 7: juntar. Los $\sum_j a_j^2$ se cancelan:

$$\sum_i \Sigma_i^2 = \sum_j a_j^2 + 2(P - W_{\text{cut}}) = S^2 - 2W_{\text{cut}}, \quad (10)$$

y reemplazando en la ec. (4),

$$\boxed{C(z) = \frac{2}{3} S^2 - 2 W_{\text{cut}}(z)} \quad (11)$$

Qué significa.

- S está fijo por la instancia, así que $\frac{2}{3}S^2$ es una constante y minimizar C equivale a maximizar W_{cut} .
- W_{cut} es un corte ponderado en tres partes: se suma $a_j a_k$ por cada par en cajas distintas. Es Max 3-Cut con peso $a_j a_k$ en la arista (j, k) .
- Todos los pesos son positivos, así que hay arista entre todos los pares: el grafo es el completo K_n .
- Como $C \geq 0$, siempre $W_{\text{cut}} \leq S^2/3$. Una partición perfecta es un corte que alcanza esa cota.

Ejemplo. La instancia de $n = 5$ de la tabla 7, $a = (2, 3, 7, 8, 10)$, tiene $S = 30$, $\sum_j a_j^2 = 226$ y $P = (900 - 226)/2 = 337$ (tabla 1).

Tabla 1: Verificación de la ec. (11) en dos particiones de $a = (2, 3, 7, 8, 10)$.

partición	sumas	C directo	pares en la misma caja	W_{cut}	$\frac{2}{3}S^2 - 2W_{\text{cut}}$
$\{10\}, \{3, 7\}, \{2, 8\}$	10, 10, 10	0	$3 \cdot 7 + 2 \cdot 8 = 37$	$337 - 37 = 300$	$600 - 600 = 0$
$\{10, 2\}, \{3, 7\}, \{8\}$	12, 10, 8	$4 + 0 + 4 = 8$	$10 \cdot 2 + 3 \cdot 7 = 41$	$337 - 41 = 296$	$600 - 592 = 8$

1.3 Codificación en qutrits

Cada número es un qutrit. El autovalor $m_j \in \{-1, 0, 1\}$ de $L_z^{(j)}$ fija la caja mediante el operador dígito $\hat{d}_j = L_z^{(j)} + I$, con autovalores $z_j = m_j + 1 \in \{0, 1, 2\}$. Como $\delta_{z_j, z_k} = \delta_{m_j, m_k}$, basta escribir la delta en términos de L_z . Revisando los nueve casos $m, m' \in \{-1, 0, 1\}$ se comprueba que

$$\delta_{m, m'} = 1 - m^2 - m'^2 + \frac{1}{2}mm' + \frac{3}{2}m^2m'^2, \quad (12)$$

de modo que el indicador de corte de un par es

$$1 - \delta_{z_j, z_k} = -\frac{1}{2} h_{jk}, \quad h_{jk} = L_z^{(j)} L_z^{(k)} - 2[(L_z^{(j)})^2 + (L_z^{(k)})^2] + 3(L_z^{(j)})^2 (L_z^{(k)})^2, \quad (13)$$

donde h_{jk} es exactamente el término de arista de la ec. (28) del paper. Reemplazando en la ec. (11),

$$H_C = 2H_{3\text{cut}} + \frac{2}{3}S^2, \quad H_{3\text{cut}} = \frac{1}{2} \sum_{j < k} a_j a_k h_{jk} = -W_{\text{cut}}. \quad (14)$$

La constante no cambia ni el estado fundamental ni los gradientes, así que MWNP es la ec. (28) del paper sobre K_n con pesos $a_j a_k$. Numéricamente, la diagonal de la ec. (2) y la de $2H_{3\text{cut}} + \frac{2}{3}S^2$ coinciden con diferencia 0.

Dos consecuencias:

- Como $E_0 = 0$, el error relativo $|E - E_0|/|E_0|$ y el approximation ratio E/E_0 del paper no están definidos. Usamos $p_{\text{éxito}}$ (sección 3.4).
- El estado de referencia y el mixer son los del paper, $H_M = -\sum_j [(L_z^{(j)})^2 + \sqrt{2}L_x^{(j)}]$, cuyo fundamental es $|+_3\rangle^{\otimes n}$, la superposición uniforme de las 3^n asignaciones.

1.4 El pool de operadores

El pool sale de los conmutadores anidados del Hamiltoniano adiabático $H_{\text{ad}}(\lambda) = (1 - \lambda)H_M + \lambda H_{3\text{cut}}$ (ecs. 12, 17 y 18 del paper), con $O_0 = \partial_\lambda H_{\text{ad}} = H_{3\text{cut}} - H_M$ y $O_k = [H_{\text{ad}}, O_{k-1}]$. Con $\ell = 1$ se usa O_1 ; con $\ell = 2$, O_1 y O_3 . Cada monomio (string de operadores de momento angular) se extrae por separado y se hermitiza, $V = (D + D^\dagger)/2$, eliminando repetidos.

$\ell = 1$: **de dónde sale $2n^2 - n$.** Como $[H_M, H_M] = [H_{3\text{cut}}, H_{3\text{cut}}] = 0$, la dependencia en λ se cancela y

$$O_1 = [H_M, H_{3\text{cut}}] = -\frac{1}{2} \sum_{j < k} w_{jk} ([X_j, h_{jk}] + [X_k, h_{jk}]), \quad X_j = (L_z^{(j)})^2 + \sqrt{2} L_x^{(j)}. \quad (15)$$

El término $(L_z^{(j)})^2$ conmuta con h_{jk} , que es diagonal. Con $[L_x, L_z] = -iL_y$ y $[L_x, L_z^2] = -i\{L_y, L_z\}$,

$$[X_j, h_{jk}] = -i\sqrt{2}(L_y^{(j)} L_z^{(k)} - 2\{L_y, L_z\}^{(j)} + 3\{L_y, L_z\}^{(j)} (L_z^{(k)})^2), \quad (16)$$

y lo mismo con $j \leftrightarrow k$ (es el Apéndice A del paper, con pesos). Los monomios distintos, ya hermitizados, son de cinco tipos (tabla 2). Los dos órdenes de $L_y L_z$ en un mismo sitio dan el mismo V , porque $(L_y L_z)^\dagger = L_z L_y$. Cada par $\{j, k\}$ aporta 4 operadores y cada sitio 1:

$$N_{\ell=1} = 4 \binom{n}{2} + n = 2n^2 - n. \quad (17)$$

El coeficiente del término de un sitio es $\propto \sum_k w_{jk} > 0$, así que no se anula. En un grafo general, el conteo es $4|E| + n$; para K_6 da 66, como en el paper.

Tabla 2: Operadores del pool $\ell = 1$ sobre K_n . El soporte es el número de qutrits sobre los que actúa.

operador V	sale de	soporte	cuántos
$\frac{1}{2}\{L_y, L_z\}^{(j)}$	$[L_x^{(j)}, (L_z^{(j)})^2]$	1	n
$L_y^{(j)} L_z^{(k)}, L_z^{(j)} L_y^{(k)}$	$[L_x, L_z^{(j)} L_z^{(k)}]$	2	$2 \binom{n}{2}$
$\frac{1}{2}\{L_y, L_z\}^{(j)} (L_z^{(k)})^2, (L_z^{(j)})^2 \frac{1}{2}\{L_y, L_z\}^{(k)}$	$[L_x, (L_z^{(j)})^2 (L_z^{(k)})^2]$	2	$2 \binom{n}{2}$
total			$2n^2 - n$

$\ell = 2$: **de dónde sale el conteo.** $O_3 = [H_{\text{ad}}, [H_{\text{ad}}, O_1]]$ depende de λ ; el pool toma todos los monomios que aparecen en algún coeficiente. El soporte crece a lo más en un sitio por conmutador con un término de dos sitios: O_1 vive en ≤ 2 sitios, O_2 en ≤ 3 y O_3 en ≤ 4 . Como K_n es simétrico bajo permutaciones de sitios, cada subconjunto de w sitios aporta el mismo número t_w de operadores, y el total es

$$N_{\ell=2} = \sum_{w=1}^4 t_w \binom{n}{w}. \quad (18)$$

De dónde salen los t_w . Cada operador del pool es un producto de factores locales, uno por sitio del soporte (tras hermitizar, el orden de los factores dentro de un sitio no importa). Si se agrupan los operadores de un soporte por su *patrón*, el multiconjunto de factores locales sin importar a qué sitio va cada uno, t_w es la suma, sobre patrones, del número de formas distintas de repartir esos factores entre los w sitios (tabla 3). Por ejemplo, el patrón (L_y, L_z, L_z, L_z) de 4 sitios se puede colocar de 4 formas (qué sitio lleva L_y), y el patrón $(L_z, L_z, L_y L_z, L_z^2)$ de $4!/2! = 12$ formas. Los patrones mismos salen de la expansión simbólica de O_1 y O_3 .

El polinomio, término a término. Con $\binom{n}{2} = \frac{n(n-1)}{2}$, $\binom{n}{3} = \frac{n(n-1)(n-2)}{6}$ y $\binom{n}{4} = \frac{n(n-1)(n-2)(n-3)}{24}$,

$$\begin{aligned} 32 \binom{n}{4} &= \frac{4}{3}(n^4 - 6n^3 + 11n^2 - 6n) = \frac{4}{3}n^4 - 8n^3 + \frac{44}{3}n^2 - 8n, \\ 78 \binom{n}{3} &= 13(n^3 - 3n^2 + 2n) = 13n^3 - 39n^2 + 26n, \\ 40 \binom{n}{2} &= 20(n^2 - n) = 20n^2 - 20n, \\ 3 \binom{n}{1} &= 3n. \end{aligned}$$

Tabla 3: Patrones del pool $\ell = 2$ por tamaño de soporte w . Se escribe yz por el factor local $L_y L_z$ (hermitizado), zz por L_z^2 , etc. “Formas” es el número de maneras de repartir los factores de un patrón entre los w sitios ordenados.

w	patrones	cuántos	formas	aporte a t_w
1	$yz; xyz; yzzz$	3	1	3
2	dos factores distintos, p.ej. $(y, z), (x, yz), (yz, zz), (xz, yzz)$	20	2	40
3	tres factores distintos, p.ej. $(x, y, z), (z, xz, yz)$	11	6	66
	con dos factores iguales: $(z, z, yz), (yz, zz, zz), (zz, zz, xyz), (zz, zz, yzzz)$	4	3	12
4	$(z, z, yz, zz), (z, zz, zz, yzz)$	2	12	24
	$(y, z, z, z), (zz, zz, zz, yzzz)$	2	4	8
t_1, t_2, t_3, t_4				3, 40, 78, 32

Sumando coeficiente por coeficiente,

$$N_{\ell=2} = \underbrace{\frac{4}{3} n^4}_{4 \text{ sitios}} + \underbrace{(-8 + 13) n^3}_{4 \text{ y } 3 \text{ sitios}} + \underbrace{\left(\frac{44}{3} - 39 + 20\right) n^2}_{-13/3} + \underbrace{(-8 + 26 - 20 + 3) n}_1 = \frac{4}{3} n^4 + 5n^3 - \frac{13}{3} n^2 + n. \quad (19)$$

El término cuártico viene sólo de los operadores de 4 sitios: hay $\binom{n}{4} \approx n^4/24$ conjuntos de 4 qutrits y cada uno aporta 32 operadores. El cúbico mezcla los de 4 sitios (el $-6n^3$ de $\binom{n}{4}$) con los de 3 sitios, y así hacia abajo. La fórmula reproduce exactamente los tamaños contados para $n = 5, \dots, 10$, y el conteo por soporte se verificó directamente en $n = 5, 6, 7$. En el rango $n = 5-10$ crece como $n^{3.65}$ porque los términos de menor grado todavía pesan; asintóticamente va como $\frac{4}{3} n^4$.

El tamaño del pool para cualquier orden ℓ

El modelo que se cuenta. El pool que construye el código es combinatorio, y conviene fijarlo con precisión. Cada operador es una *palabra* de factores $L_a^{(j)}$, $a \in \{x, y, z\}$. Factores de sitios distintos conmutan; dentro de un sitio se conserva el orden. El conmutador se calcula con la regla de Leibniz y $[L_x, L_y] = iL_z$ (y cíclicas), sin usar identidades propias de espín 1 como $L_z^3 = L_z$. El pool de orden ℓ son las palabras con coeficiente no nulo en $O_1, O_3, \dots, O_{2\ell-1}$, como polinomios en λ , identificadas por su multiconjunto de factores. Escribimos (a, b, c) para el número de L_x, L_y y L_z que una palabra tiene en un sitio.

Proposición 1 (forma general). Para $n \geq 2$,

$$N_\ell(n) = \sum_{w=1}^{2\ell} t_w^{(\ell)} \binom{n}{w}, \quad (20)$$

donde $t_w^{(\ell)}$, el número de operadores con soporte en un conjunto dado de w qutrits, no depende de n (salvo eventualmente en un conjunto finito de valores de n , por cancelaciones accidentales). Como $\binom{n}{w} = 0$ si $w > n$, la fórmula vale también para $n < 2\ell$; no hace falta pedir $n \geq 2\ell$. Para $n = 1$ el pool es vacío, porque $H_C = 0$.

Demostración.

1. *El soporte nunca se achica.* $[L_a^{(s)}, \cdot]$ sólo reemplaza un factor del sitio s por otro del mismo sitio, y los demás factores de un término se multiplican. Por eso el soporte de cada palabra de $[T, M]$ contiene al de M y al de T .
2. *El soporte crece a lo más en uno por conmutador.* Factores de sitios disjuntos conmutan, así que $[T, M] \neq 0$ exige que T comparta un sitio con M . Como los términos de H_{ad} tienen a lo más dos sitios, cada conmutador agrega a lo más uno. Además $O_1 = [H_{\text{ad}}, H_C - H_M] = [H_M, H_C]$, porque los términos en λ se cancelan, y vive en ≤ 2 sitios. Luego O_k vive en $\leq k + 1$ sitios, y el pool de orden ℓ en $\leq 2\ell$.

3. Las palabras con soporte S sólo ven lo que pasa dentro de S . Por el paso 1, sólo contribuyen los términos de H_{ad} contenidos en S . De los pares $\{j, m\}$ con $m \notin S$ sobrevive únicamente su parte de un sitio, $-2(L_z^{(j)})^2$, que cambia el coeficiente de $(L_z^{(j)})^2$ a $\mu = 3 - 2n$ en O_0 y a $-1 + \lambda\mu$ en H_{ad} . El conteo dentro de S es entonces el de K_w , con μ como parámetro. Los coeficientes de las palabras son polinomios en (λ, μ) ; uno que no es idénticamente nulo sólo se anula para finitos valores de μ , es decir, de n . Por la simetría de K_n , todos los subconjuntos de w sitios dan el mismo $t_w^{(\ell)}$. $\square$

Numéricamente, los $t_w^{(\ell)}$ de $\ell = 2, 3$ son idénticos para $n = w, \dots, 60$.

Proposición 2 (operadores de un qutrit). Un operador de un sitio (a, b, c) está en el pool de orden ℓ si y sólo si b y c son impares y $2a + 2b + c \leq 2\ell + 1$. En consecuencia

$$t_1^{(\ell)} = \#\{(a, \beta, \gamma) \in \mathbb{N}^3 : a + 2\beta + \gamma \leq \ell - 1\} = \left\lfloor \frac{(\ell + 1)(\ell + 3)(2\ell + 1)}{24} \right\rfloor, \quad (21)$$

que da 1, 3, 7, 13, 22, 34, $\dots$ para $\ell = 1, 2, \dots$, y crece como $\ell^3/12$.

Demostración. Por el paso 1 de la Proposición 1, las palabras de un sitio sólo provienen de palabras de un sitio y de los términos de un sitio. El problema se reduce a un qutrit: se parte de $O_0 = \mu L_z^2 + \sqrt{2}L_x$ y se aplica ad_A , con $A = \alpha L_z^2 + \beta L_x$, $\alpha = -1 + \lambda\mu$ y $\beta = -\sqrt{2}(1 - \lambda)$.

- *Lo que cambia en cada paso.* ad_{L_x} cambia una y por una z o una z por una y . $\text{ad}_{L_z^2}$ cambia una x por una y o al revés, y agrega una z . En ambos casos b y c cambian de paridad, y $\varphi = 2a + 2b + c$ sube a lo más en 1. Como O_0 tiene $b = c = 0$ y $\varphi = 2$, toda palabra de O_k tiene $b \equiv c \equiv k \pmod{2}$ y $\varphi \leq k + 2$.
- *Necesidad.* Si (a, b, c) aparece en algún O_k con $k \leq 2\ell - 1$ impar, entonces b y c son impares y $\varphi \leq 2\ell + 1$.
- *Suficiencia.* Hay tres movimientos que suben φ en exactamente 1: $Z: z \rightarrow y$, que lleva $(a, b, c) \mapsto (a, b + 1, c - 1)$; $S_2: y \rightarrow x$ más una z , $(a + 1, b - 1, c + 1)$; y $S_1: x \rightarrow y$ más una z , $(a - 1, b + 1, c + 1)$. Sea (a, b, c) con $b = 2\beta + 1$, $c = 2\gamma + 1$. Partiendo de $L_z^2 = (0, 0, 2)$:

- Z lleva a $(0, 1, 1)$;
- $(S_2 S_1)^{\gamma + \beta}$ lleva a $(0, 1, 1 + 2\gamma + 2\beta)$;
- $Z^{2\beta}$ lleva a $(0, 1 + 2\beta, 1 + 2\gamma)$;
- $(S_2 Z)^a$ lleva a (a, b, c) .

Son $1 + 2(\gamma + \beta) + 2\beta + 2a = \varphi - 2$ pasos, así que (a, b, c) aparece en $O_{\varphi - 2}$, con $\varphi - 2 \leq 2\ell - 1$ impar.

- *No hay cancelación.* En ese nivel superior todos los caminos usan el mismo número de pasos de cada tipo, así que todas sus contribuciones a una misma palabra tienen la misma fase y peso positivo. Los dos términos de partida dan potencias distintas, $\alpha^s \beta^{k-s}$ y $\alpha^{s+1} \beta^{k-s-1}$, y α y β son polinomios en λ linealmente independientes salvo si $\mu = 1$, es decir, salvo si $n = 1$.

El conteo de $a + 2\beta + \gamma \leq L$ da $(q + 1)(q + 2)(4q + 3)/6$ si $L = 2q$ y $(q + 1)(q + 2)(4q + 9)/6$ si $L = 2q + 1$, que es la expresión con piso de la ec. (21). $\square$

Proposición 3 (soporte máximo).

$$t_{2\ell}^{(\ell)} = 2\ell \cdot 2^{2\ell - 1} = \ell 4^\ell, \quad (22)$$

es decir 4, 32, 192, 1024, $\dots$, y por lo tanto $N_\ell(n) = \ell 4^\ell n^{2\ell} / (2\ell)! + O(n^{2\ell - 1})$.

Demostración. Una palabra de $O_{2\ell - 1}$ con 2ℓ sitios exige que cada uno de los $2\ell - 2$ conmutadores posteriores a O_1 agregue un sitio nuevo q . Lo hace con un término de par $\{p, q\}$, con p ya presente. Hay dos posibilidades:

- $[L_z^{(p)} L_z^{(q)}, M] = [L_z^{(p)}, M] L_z^{(q)}$: el sitio nuevo recibe z ;
- el término $(L_z^{(p)})^2 (L_z^{(q)})^2$: el sitio nuevo recibe zz , y p una z adicional.

En ambos casos se aplica ad_{L_z} al sitio p , que anula todo sitio que sólo tiene factores z . Las palabras de dos sitios de $O_1 = [H_M, H_C]$ son “y en un sitio, z en el otro” o “yz en uno, zz en el otro”. Por lo tanto sólo el sitio con la y (la raíz) puede recibir sitios nuevos, y la palabra final es una estrella:

- la raíz lleva una y (cada ad_{L_z} alterna $y \leftrightarrow x$, y son un número par) y tantas z como hojas de tipo zz ;
- cada una de las $2\ell - 1$ hojas es z o zz .

El multiconjunto queda determinado por la raíz (2ℓ opciones) y el tipo de cada hoja ($2^{2\ell-1}$ opciones). Todas las contribuciones a una misma palabra tienen la misma fase y coeficiente positivo, así que no se cancelan. $\square$

Tabla 4: Operadores por subconjunto de w qutrits, $t_w^{(\ell)}$. En negrita, los valores que coinciden con las Proposiciones 2 y 3. Los de $\ell \leq 3$ salen de la expansión simbólica, y los de $\ell \geq 4$ del enumerador numérico (`cluster/pool_numerico.py`), que reproduce exactamente a la simbólica en $\ell \leq 3$. “.”: no calculado (los t_w intermedios de $\ell \geq 5$ superan la memoria de BitWit con este método). La última columna es $t_{2\ell}$ cuando $2\ell > 8$.

ℓ	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	$t_{2\ell}$
1	1	4							
2	3	40	78	32					
3	7	208	1156	2132	1270	192			
4	13	756	8489	35444	63990	48110	13174	1024	
5	22	2202	41602	.	.	.	.	.	5120
6	34	5442	.	.	.	.	.	.	.

Los valores calculados confirman las dos fórmulas cerradas fuera de los casos usados para encontrarlas: $t_1^{(5)} = 22$ y $t_1^{(6)} = 34$, y $t_8^{(4)} = 1024 = 4 \cdot 4^4$ y $t_{10}^{(5)} = 5120 = 5 \cdot 4^5$. Con la tabla completa de $\ell = 4$, $N_4(n) = 13n + 756\binom{n}{2} + 8489\binom{n}{3} + 35444\binom{n}{4} + 63990\binom{n}{5} + 48110\binom{n}{6} + 13174\binom{n}{7} + 1024\binom{n}{8}$. Los $t_w^{(\ell)}$ intermedios ($2 \leq w \leq 2\ell - 1$) no tienen todavía fórmula cerrada. Se pueden atacar igual que el soporte máximo, clasificando las palabras según cuántos conmutadores no agregan sitio; queda como trabajo pendiente.

Los pesos no cambian el pool. El pool se construye una vez por n con K_n sin pesos y se reutiliza para todas las instancias. Para descartar cancelaciones que sólo ocurran con pesos iguales, lo reconstruimos con pesos genéricos (primos distintos por arista) en $n = 4$ y 5 , con $\ell = 1$ y $\ell = 2$: el conjunto de operadores es idéntico.

Duplicados en el pool $\ell = 2$. El código arma el pool $\ell = 2$ como la lista de O_1 seguida de la de O_3 , y todos los operadores de O_1 vuelven a aparecer en O_3 . La lista tiene entonces $N_{\ell=1}$ entradas repetidas (tabla 5). Esto no cambia ningún resultado: la copia tiene un índice mayor y `argmax` elige siempre la primera. Sí infla el tamaño reportado del pool y el número de mediciones de gradiente (ver sección 6).

Tabla 5: Tamaño del pool de K_n . “Lista” es la longitud de la lista que usa el código, con los operadores de O_1 repetidos.

n	$N_{\ell=1}$	$N_{\ell=2}$ (distintos)	lista $\ell = 2$	$N_{\ell=2}/N_{\ell=1}$
5	45	1355	1400	30
6	66	2658	2724	40
7	91	4711	4802	52
8	120	7752	7872	65
9	153	12051	12204	79
10	190	17910	18100	94

2 Implementación

2.1 El algoritmo

1. Preparar $|\psi_0\rangle = |+_3\rangle^{\otimes n}$ y fijar $k = 0$.

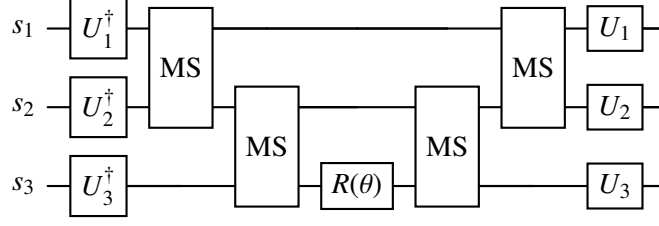


Figura 2: Compilación de $e^{-i\theta V}$ para un generador de soporte $w = 3$ que factoriza (modelo del Apéndice B del paper): diagonalización local, escalera de $2(w - 1) = 4$ compuertas MS con una rotación parametrizada al centro, y deshacer la diagonalización.

- **Gradiente del pool (paso 2).** Con H_C diagonal, $g_j = -2 \operatorname{Im} \langle V_j \psi | H_C \psi \rangle$: una aplicación de V_j por operador. Un barrido completo cuesta $\sum_j 3^{n+w_j}$.
- **Gradiente de la energía (paso 5), método adjunto.** Con $|\phi_m\rangle = U_m \cdots U_1 |\psi_0\rangle$ y $|\lambda_m\rangle = U_{m+1}^\dagger \cdots U_k^\dagger H_C |\psi\rangle$,

$$\frac{\partial E}{\partial \theta_m} = 2 \operatorname{Im} \langle \lambda_m | V_m | \phi_m \rangle. \quad (25)$$

Una pasada hacia adelante y otra hacia atrás dan los k componentes con unas $3k$ aplicaciones de bloques: el costo de unas tres evaluaciones de la energía, en vez de $k + 1$ con diferencias finitas. Se entrega a BFGS (scipy, jac=True, gtol= 10^{-10} , hasta 1000 iteraciones). Coincide con diferencias finitas a 10^{-8} .

- **Memoria.** El estado más los $k + 1$ estados intermedios de la pasada hacia adelante: en $n = 9$, $k = 150$, unos 50 MB.

Validación. J. Molina guardó los operadores y parámetros finales de sus 40 corridas. Reconstruimos esos 40 estados con nuestro simulador, y las energías coinciden con las suyas con diferencia máxima 7.7×10^{-12} . Además, corriendo nuestro ADAPT sobre sus instancias con sus mismos ajustes, $p_{\text{éxito}}$ coincide (± 0.05) en 35 de 40 casos.

3 Protocolo

3.1 Instancias

Cada instancia cumple tres condiciones, que se verifican por enumeración de las 3^n asignaciones:

1. **Enteros distintos.** Números repetidos agregan degeneración: dos números iguales intercambiados dan otra asignación óptima. Eso sube la probabilidad de acertar por azar y hace la instancia más fácil de un modo que no depende del algoritmo.
2. **Partición perfecta.** Garantiza $E_0 = 0$ conocido. Son las instancias “balanceadas” del trabajo de J. Molina.
3. **Solución única** (degeneración exactamente 6). Es el caso más exigente para cada n : la probabilidad de acertar midiendo la superposición uniforme es la mínima posible, $p_{\text{éxito}}^{\text{azar}} = 6/3^n$, y la métrica de éxito queda sin ambigüedad.

Por qué el rango $[1, 3n]$. El rango tiene que ser $\geq n$ para que quepan n números distintos. Con un rango chico las sumas coinciden a menudo y aparecen varias particiones perfectas, lo que rompe la unicidad. Con un rango grande, la partición perfecta se vuelve rara y la escala de energía de H_C crece como R^2 . La tabla 6 muestra, para 10000 sorteos por celda, qué fracción de conjuntos de números distintos cumple las tres condiciones:

- Con el rango fijo $[1, 15]$ de J. Molina, en $n = 10$ no aparece *ninguna* instancia válida (0.0 %).

- Con $[1, 2n]$ la fracción cae a 1.1 % en $n = 10$ y va en bajada: con números tan chicos muchas sumas coinciden y la solución deja de ser única.
- Con $[1, 3n]$ la fracción se mantiene entre 3.3 y 11.4 % en todo $n = 5-10$: hay que sortear entre ~ 10 y ~ 30 conjuntos por instancia aceptada. Rangos mayores también sirven, con números más grandes (tabla 6).
- No hay un rango “correcto”: elegimos el menor de los probados que se mantiene estable al crecer n , para tener números chicos (y energías chicas) sin quedarnos sin instancias.

Queda la pregunta de si el rango elegido sesga la dificultad. Lo probamos en $n = 6$: las 9 instancias válidas que existen con números en $[1, 8]$ no se resuelven con más frecuencia que las de $[1, 18]$ (sección 4.11). Dentro de lo que medimos, el rango decide cuántas instancias válidas hay, no cuán difíciles son.

Tabla 6: Porcentaje de conjuntos de n enteros distintos sorteados en $[1, R]$ que cumplen las tres condiciones (partición perfecta única). Cada celda es una estimación de Monte Carlo con 10000 sorteos, con error estándar de a lo más ± 0.4 puntos. En $n = 5$ las columnas $R = 3n$ y $R = 15$ son el mismo rango y se calculan con los mismos sorteos. En negrita, el rango usado.

n	$R = 2n$	$R = 3n$	$R = 4n$	$R = 6n$	$R = 15$
5	7.9	3.3	1.2	0.7	3.3
6	12.6	5.2	2.5	1.2	7.7
7	15.8	7.8	4.6	2.0	14.9
8	15.2	10.7	6.8	3.5	15.3
9	8.2	11.4	9.5	6.2	3.7
10	1.1	6.9	9.5	8.3	0.0

Las instancias se generan de forma determinista por (semilla, n , índice): la instancia 7 de $n = 8$ es siempre la misma. Usamos 20 por tamaño para $n = 5-9$. La tabla 7 muestra una de cada tamaño.

Tabla 7: Una instancia por tamaño y su partición óptima.

n	rango	$p_{\text{éxito}}^{\text{azar}}$	números	cajas óptimas
5	[1, 15]	2.5×10^{-2}	[2, 3, 7, 8, 10]	10 3, 7 2, 8
6	[1, 18]	8.2×10^{-3}	[1, 6, 8, 10, 12, 17]	8, 10 6, 12 1, 17
7	[1, 21]	2.7×10^{-3}	[1, 4, 6, 10, 13, 16, 19]	10, 13 4, 19 1, 6, 16
8	[1, 24]	9.1×10^{-4}	[8, 9, 10, 11, 12, 14, 16, 19]	14, 19 10, 11, 12 8, 9, 16
9	[1, 27]	3.0×10^{-4}	[1, 2, 5, 6, 7, 8, 20, 21, 26]	6, 26 5, 7, 20 1, 2, 8, 21
10	[1, 30]	1.0×10^{-4}	[2, 3, 12, 15, 16, 17, 18, 22, 27, 30]	12, 15, 27 3, 16, 17, 18 2, 22, 30

3.2 Órdenes

Qué número se asigna a qué qutrit es arbitrario y no cambia el problema: permutar los qutrits permuta las cadenas, pero la solución es la misma. Lo que sí cambia es la *numeración* de los operadores del pool, porque la lista está ordenada por índice de qutrit.

Por qué eso importa. En el paso 4, ADAPT elige el operador de mayor $|g_j|$. Desde $|+3\rangle^{\otimes n}$ hay operadores con exactamente el mismo gradiente por simetría: en todas nuestras corridas hay empates en la primera iteración (la mediana es de 4 operadores empatados). Ante un empate, argmax toma el primero de la lista, y cuál es el primero depende de qué número quedó en qué qutrit.

Ejemplo real ($n = 6$, $\ell = 1$, $a = \{2, 3, 5, 7, 9, 10\}$). En $k = 1$ empatan $L_z^{(9)} L_y^{(10)}$ y $L_y^{(9)} L_z^{(10)}$, es decir, L_z sobre el qutrit que lleva el 9 y L_y sobre el que lleva el 10, o al revés; ambos actúan sobre el par de mayor producto, $9 \cdot 10$. En los 5 órdenes, ADAPT tomó el que pone L_z en el qutrit de menor índice. Cuando el 10 quedó antes que el 9 (órdenes 1 y 4), el primer operador fue $L_z^{(10)} L_y^{(9)}$ y la corrida resolvió la instancia

($p_{\text{éxito}} = 0.49$). Cuando el 9 quedó antes (órdenes 0, 2 y 3), fue $L_z^{(9)}L_y^{(10)}$ y la corrida falló ($p_{\text{éxito}} \approx 0$). Un único empate, decidido por una numeración arbitraria, separa el éxito del fracaso.

Qué significa en la práctica.

- Una corrida sola no caracteriza a una instancia: con otra numeración de los qutrits el resultado puede ser el opuesto.
- Por eso cada instancia se corre bajo 5 órdenes de sus números (el ascendente y 4 permutaciones aleatorias deterministas), y se reporta la fracción de órdenes que la resuelven.
- En un experimento pasaría lo mismo: el resultado dependería de cómo se cableen los números en el procesador. Habría que promediar sobre órdenes o fijar un desempate explícito, por ejemplo al azar. En el paper conviene mencionarlo.

3.3 Ajustes y estrategias de inicialización

$\epsilon = 10^{-3}$ sobre la norma del gradiente, en la escala de la ec. (2), y tope de 150 parámetros. Comparamos tres estrategias:

- **warm:** $\theta_{k+1}^{(0)} = (\theta_k^*, 0)$. Es el ADAPT estándar del paper.
- **(ii) ADAPT desde $\theta = 0$:** $\theta_{k+1}^{(0)} = 0$ en cada iteración, es decir, se reoptimiza desde $|+_3\rangle^{\otimes n}$. Como el estado optimizado cambia, el paso 2 elige otros operadores: es otro algoritmo, y es el que pone a prueba la conjetura.
- **(i) secuencia fija desde $\theta = 0$:** se toma la secuencia elegida por warm y, para cada k , se reoptimiza ese circuito desde 0. Aísla el efecto del punto de partida sobre un circuito dado. Es el “cold start” de las Figs. 5–6 del paper.

3.4 Análisis de datos

Métrica. $p_{\text{éxito}} = \sum_{z \in \text{óptimos}} |\langle z | \psi \rangle|^2$ es la probabilidad de que una medición entregue una partición óptima. Una corrida *resuelve* la instancia si $p_{\text{éxito}} \geq 0.1$. La probabilidad de ver la solución al menos una vez en M mediciones es $1 - (1 - p_{\text{éxito}})^M$, que con $p_{\text{éxito}} = 0.1$ y $M = 50$ ya es 0.995. Verificar cuál de las cadenas medidas es óptima es trivial en un computador clásico. Un criterio más estricto, exigir que la cadena más probable sea la correcta, cuenta como falla un estado con la mitad de su peso sobre la solución.

Tasa por instancia. Las 5 corridas de una misma instancia (una por orden) no son independientes: comparten los números. Tratar las 100 corridas de un n como independientes exageraría la significancia. La unidad estadística es la instancia i , con tasa $\tau_i = \frac{1}{5} \sum_{o=1}^5 \mathbb{I}[p_{\text{éxito}}^{(i,o)} \geq 0.1]$. La tasa de un n es $\bar{\tau} = \frac{1}{20} \sum_i \tau_i$.

Intervalos de confianza. Bootstrap sobre instancias: se remuestrean las 20 instancias con reemplazo 10^4 veces, y el IC del 95 % son los percentiles 2.5 y 97.5 de $\bar{\tau}$.

Comparaciones pareadas. Para comparar dos variantes A y B (dos pools o dos estrategias) sobre las mismas instancias y órdenes, se cuenta en cuántas instancias $\tau_i^A > \tau_i^B$ y en cuántas al revés, descartando los empates. Bajo la hipótesis nula de que ninguna es mejor, ese conteo es binomial con probabilidad 1/2. El valor p es el de la prueba de signo bilateral. No supone ninguna distribución para τ_i .

Respuestas perdidas. Una corrida que falla tuvo la respuesta “al alcance” si $\max_k p_{\text{éxito}}(k) \geq 0.1$ en alguna iteración intermedia.

4 Resultados

4.1 $\ell = 1$ contra $\ell = 2$

Esta sección y la siguiente cubren $n = 5-9$ con tope de 150 parámetros. Los resultados de $\ell = 1$ contra $\ell = 2$ para $n = 10$ y 12, con tope de 80, están en la sección 4.4 (tabla 11 y fig. 6).

Tabla 8: Warm start, 20 instancias $\times$ 5 órdenes por fila. “Siempre / depende / nunca” cuenta las instancias resueltas en los 5 órdenes, en algunos o en ninguno. “Conv.” son las corridas que terminaron por el criterio de gradiente y no por el tope. k , nativas, MS y tiempo son medianas; las compuertas se cuentan como en la fig. 2.

n	ℓ	tasa	IC 95 %	siempre / depende / nunca	conv.	k	nativas	MS	tiempo
5	1	0.83	[0.68, 0.95]	13 / 5 / 2	100/100	10	129	18	0.2 s
	2	0.94	[0.89, 0.98]	15 / 5 / 0	100/100	13	216	34	1.0 s
6	1	0.67	[0.51, 0.82]	9 / 8 / 3	100/100	20	261	36	0.7 s
	2	0.85	[0.70, 0.97]	15 / 3 / 2	98/100	24	429	78	4.9 s
7	1	0.49	[0.31, 0.67]	6 / 8 / 6	100/100	18	227	32	1.1 s
	2	0.59	[0.41, 0.76]	8 / 8 / 4	91/100	42	776	134	21.2 s
8	1	0.25	[0.09, 0.43]	3 / 4 / 13	92/100	25	325	46	4.1 s
	2	0.57	[0.38, 0.75]	8 / 6 / 6	59/100	113	2137	384	6.3 min
9	1	0.13	[0.02, 0.28]	2 / 3 / 15	87/100	39	508	71	51.2 s
	2	0.22	[0.10, 0.36]	1 / 8 / 11	64/100	84	1608	289	8.6 min

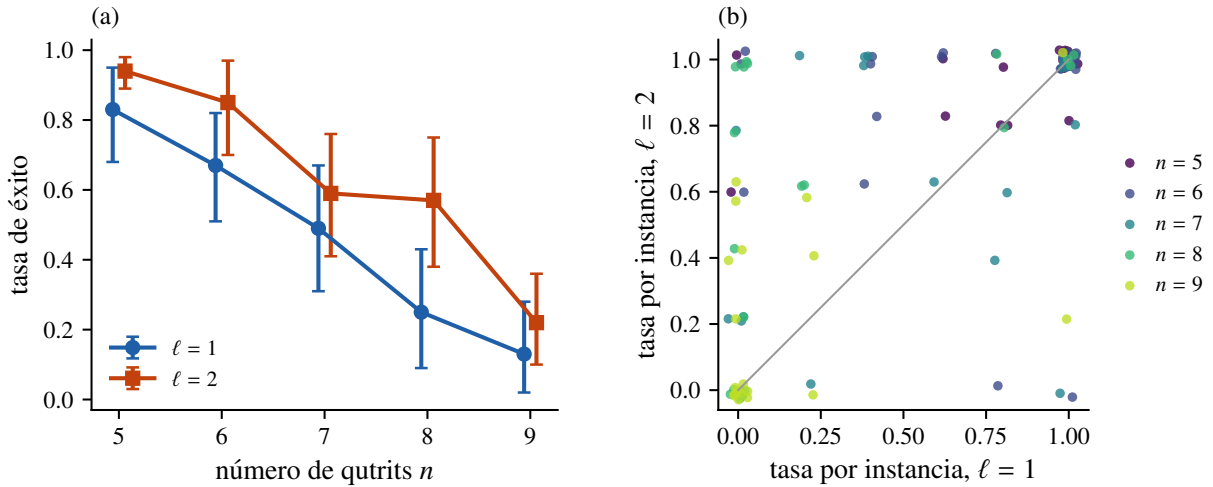


Figura 3: (a) Tasa de éxito $\bar{\tau}$ contra n , con IC del 95 %. (b) τ_i con $\ell = 1$ contra $\ell = 2$, una instancia por punto: sobre la diagonal gana $\ell = 2$. Los puntos llevan un pequeño desplazamiento aleatorio para separar los superpuestos.

Resultados.

- $\ell = 2$ supera a $\ell = 1$ en todo n (tabla 8, fig. 3a). Por instancia, $\ell = 2$ gana en 40, $\ell = 1$ en 10, y el resto empata ($p = 2.4 \times 10^{-5}$). La mayor diferencia se da en $n = 8$.
- La tasa cae con n para ambos pools. En $n = 9$ ninguno pasa de un cuarto.
- Con $\ell = 1$ la fracción resuelta se estanca hacia $k \approx 30$; con $\ell = 2$ sigue subiendo hasta $k \approx 50$ ($n \leq 7$), $k \approx 80$ ($n = 8$) y $k \approx 110$ ($n = 9$) (fig. 4).
- Con $\ell = 2$, en $n = 8$ y 9 sólo 59 y 64 de 100 corridas convergen antes del tope de 150.
- $\ell = 2$ cuesta entre 1.6 y 6.6 veces más compuertas nativas que $\ell = 1$, y su pool es de 30 a 80 veces más grande (tabla 5).

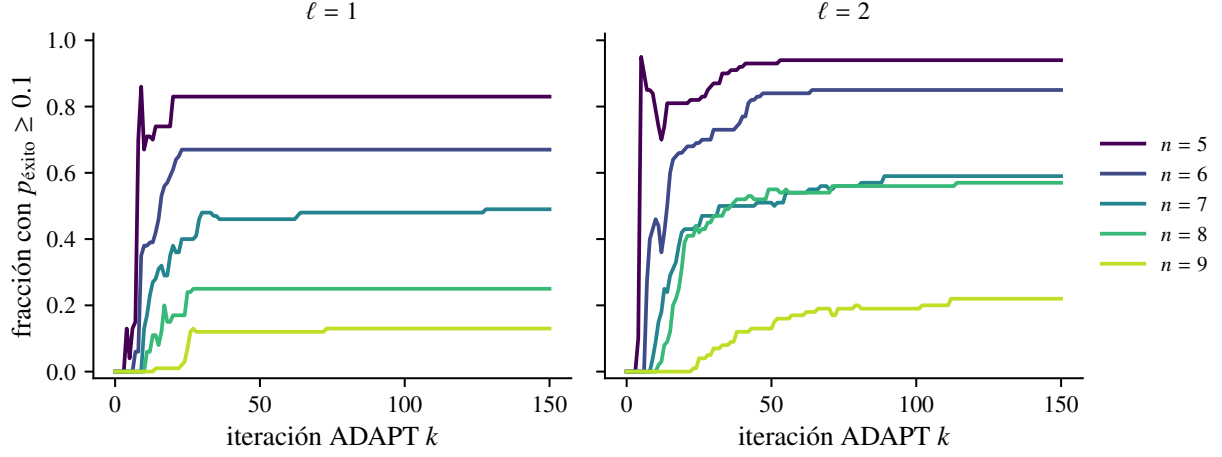


Figura 4: Fracción de corridas con $p_{\text{éxito}} \geq 0.1$ en cada iteración ADAPT, 100 corridas por curva. Una corrida que ya terminó conserva su valor final.

Conclusiones. Los operadores de tres y cuatro sitios de O_3 son los que permiten seguir mejorando cuando $\ell = 1$ se estanca. Es el mismo comportamiento que la Fig. 4b del paper muestra para Max 3-Cut. El precio es un pool mucho mayor (más mediciones de gradiente) y circuitos más caros. Como la curva de $\ell = 2$ ya se aplanó en $k = 150$, subir el tope probablemente dé ganancias modestas.

4.2 Warm start contra $\theta = 0$

Tabla 9: Tasa de éxito $\bar{\tau}$ de cada estrategia y comparación por instancia contra warm, en la forma “gana warm / gana la otra / igual”. La fila “todos” suma sobre n y da la prueba de signo.

ℓ	n	tasa			por instancia, contra warm	
		warm	(ii)	(i)	(ii) ADAPT desde 0	(i) secuencia fija
1	5	0.83	0.80	0.81	4 / 2 / 14	1 / 0 / 19
	6	0.67	0.78	0.70	1 / 7 / 12	2 / 2 / 16
	7	0.49	0.46	0.42	6 / 3 / 11	3 / 1 / 16
	8	0.25	0.20	0.21	4 / 1 / 15	2 / 0 / 18
	9	0.13	0.07	0.09	4 / 1 / 15	3 / 1 / 16
todos					19 / 14 ($p = 4.9 \times 10^{-1}$)	11 / 4 ($p = 1.2 \times 10^{-1}$)
2	5	0.94	0.91	0.92	4 / 1 / 15	2 / 0 / 18
	6	0.85	0.69	0.82	8 / 1 / 11	3 / 0 / 17
	7	0.59	0.57	0.44	7 / 6 / 7	9 / 3 / 8
	8	0.57	0.35	0.31	11 / 0 / 9	12 / 2 / 6
	9	0.22	0.09	0.12	8 / 1 / 11	6 / 3 / 11
todos					38 / 9 ($p = 2.5 \times 10^{-5}$)	32 / 8 ($p = 1.8 \times 10^{-4}$)

Resultados.

- **(ii) con $\ell = 2$.** Warm resuelve más en todo n ; la diferencia es grande en $n = 6, 8$ y 9 y mínima en $n = 7$ (tabla 9). Por instancia gana warm en 38 y (ii) en 9 ($p = 2.5 \times 10^{-5}$).
- **(ii) con $\ell = 1$.** No hay diferencia significativa: 19 contra 14 ($p = 4.9 \times 10^{-1}$). El signo cambia con n ; (ii) gana en $n = 6$, que es lo que se había visto con 10 instancias y un solo orden.
- **Mecanismo, con $\ell = 2$.** La desventaja de (ii) no es de presupuesto. Termina por el criterio de gradiente en 498/500 corridas, contra 412/500 de warm, y con circuitos más cortos (k mediano 44 contra 113).

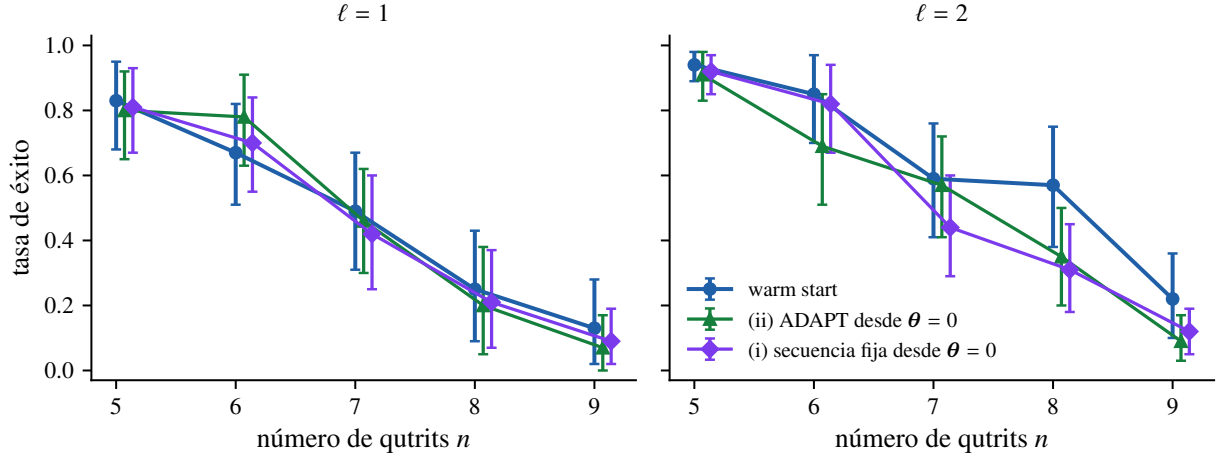


Figura 5: Tasa de éxito de las estrategias contra n , con IC del 95 %.

en $n = 8$). Encuentra la solución a mitad de camino casi tan seguido como warm: en $n = 8$ superan $p_{\text{éxito}} = 0.1$ en algún momento el 0.59 de sus corridas contra el 0.63 de warm. Pero la pierde más: termina resuelta en 0.35 contra 0.57.

- **(i) con $\ell = 2$.** Sobre el mismo circuito, partir del óptimo anterior es mejor que partir de cero: warm gana en 32 instancias y (i) en 8 ($p = 1.8 \times 10^{-4}$). La diferencia se concentra en $n \geq 7$. Reoptimizar desde cero termina con menor energía que warm sólo en el 1–13 % de las corridas, según n .
- **(i) con $\ell = 1$.** No hay diferencia significativa: 11 contra 4 ($p = 1.2 \times 10^{-1}$), con la mayoría de las instancias empatadas. Aun así, (i) termina con menor energía sólo en el 1–18 % de las corridas.

Conclusiones. La conjetura de que partir del óptimo anterior es una desventaja no se sostiene. Con $\ell = 2$ ocurre lo contrario, y por dos vías independientes:

- *En la selección de operadores (ii):* al reoptimizar desde cero, BFGS cae en puntos donde todo el pool tiene gradiente pequeño, y el criterio de parada detiene ADAPT antes.
- *En la optimización de un circuito dado (i):* con los mismos operadores, partir del óptimo anterior lleva a mínimos más bajos que partir de cero. Es el análogo de las Figs. 5–6 del paper.

Ambas son el mecanismo de *burrowing* (Grimsley *et al.*) que el paper invoca para Max 3-Cut: cada nuevo operador deforma el paisaje y warm sigue descendiendo desde donde estaba. Con $\ell = 1$ las estrategias son equivalentes en tasa de éxito, aunque reoptimizar desde cero rara vez llega a menor energía que warm.

4.3 Comparación con QAOA

J. Molina corrió QAOA para qutrits sobre sus 20 instancias de $n = 5, 6$: mixer $\sum_j L_x^{(j)}$, estado inicial $|+_3\rangle^{\otimes n}$, $p \leq 10$ capas, CMA-ES con 25 reinicios y parámetros sorteados en $[-\pi, \pi]$. Reconstruimos sus estados a partir de los parámetros guardados; las energías coinciden con las suyas a 1.6×10^{-12} .

Resultados.

- Su energía óptima *sube* al agregar capas en 116 de 180 pasos. Eso no puede ocurrir con un optimizador que converja, porque el circuito de p capas contiene al de $p - 1$ (basta anular la última capa).
- La causa es de escala. Las energías de H_C son del orden de cientos, así que el γ útil es $\sim 10^{-2}$ (del orden de $1/\text{std}(H_C)$). Un sorteo en $[-\pi, \pi]$ cae casi siempre en una región donde $e^{-i\gamma H_C}$ oscila demasiado rápido.

- Lo reoptimizamos con el mismo circuito: barrido en grilla de (γ, β) para $p = 1$, luego inicialización INTERP (Zhou *et al.*, PRX 2020) para pasar de p a $p + 1$, y BFGS. Así la energía baja monótonamente, entre 3 y 28 veces de $p = 1$ a $p = 10$.
- El QAOA de Max 3-Cut del paper (L-BFGS-B) no tiene este problema: su mejor energía baja monótonamente con p en los 8 grafos.

Tabla 10: Instancias de J. Molina (10 por n , genéricas y balanceadas): $p_{\text{éxito}}$ mediana y, entre paréntesis, instancias con $p_{\text{éxito}} \geq 0.1$. QAOA con $p = 10$ (20 parámetros). ADAPT con warm start y tope de 30 parámetros, como en sus corridas; k es la mediana de parámetros usados.

n	QAOA (J. Molina)	QAOA reoptimizado	ADAPT $\ell = 1$	k	ADAPT $\ell = 2$	k
5	0.04 (2/10)	0.44 (10/10)	0.97 (8/10)	28	1.00 (10/10)	14
6	0.01 (0/10)	0.11 (5/10)	0.64 (10/10)	14	0.97 (10/10)	18

Conclusiones. Con un número similar de parámetros, ADAPT $\ell = 2$ supera a un QAOA bien optimizado (tabla 10). La conclusión cualitativa del cuaderno de J. Molina se sostiene, pero la brecha es menor que la que muestran sus números. Para el paper, la comparación con QAOA en MWNP tiene que hacerse con un QAOA bien inicializado.

4.4 Benchmark ampliado: ADAPT ($k \leq 80$) contra QAOA ($p = 40$)

Este benchmark es el que pidió el investigador principal: 20 conjuntos de números por n , ADAPT con $\ell = 1$ y $\ell = 2$ hasta 80 iteraciones, y QAOA con $p = 40$ capas (80 parámetros) y 10 reinicios aleatorios, guardando $p_{\text{éxito}}$ en cada caso. Aquí van $n = 5-12$; falta sólo INTERP para $n \geq 10$, que está corriendo en BitWit.

Protocolo.

- **ADAPT:** warm start, $\epsilon = 10^{-3}$, tope de 80 parámetros y 5 órdenes por instancia, en total 1600 corridas. Para $n \leq 9$ se usan las primeras 80 iteraciones de las corridas con tope 150, que son idénticas a una corrida con tope 80 porque ADAPT es determinista y el tope sólo la corta. Para $n \geq 10$ las corridas son nuevas, y con $\ell = 2$ usan el barrido por soporte, que da el mismo gradiente hasta 10^{-13} . Los empates exactos se siguen resolviendo por redondeo, como en el resto del informe.
- **QAOA:** el circuito del código del paper para Max 3-Cut, con mezclador $\sum_j L_x^{(j)}$ y estado inicial $|+3\rangle^{\otimes n}$. Lo elegimos por consistencia con los resultados de QAOA ya publicados. Con este mezclador, $|+3\rangle^{\otimes n}$ no es su estado fundamental, así que el circuito no es una versión digitalizada de la evolución adiabática. El mezclador de la ec. (9) del paper, $-\sum_j [(L_z^{(j)})^2 + \sqrt{2}L_x^{(j)}]$, sí lo es; correrlo queda pendiente. H_C se normaliza por su desviación estándar, se toman 10 puntos iniciales uniformes en $[-\pi, \pi]^{80}$ y se optimiza con L-BFGS-B (cotas $\pm\pi$, gradiente analítico, hasta 1000 iteraciones), en total 1600 reinicios. “Mejor de 10” es el reinicio de menor energía, el que elegiría un experimento sin conocer la solución. Como referencia más fuerte se agrega la cadena INTERP: grilla en $p = 1$ y, de p a $p + 1$, los óptimos interpolados como punto inicial.

Resultados.

- **Contra QAOA con reinicios aleatorios, ADAPT gana en todo n y en toda métrica** (tabla 11, fig. 6). En $n = 12$, el $p_{\text{éxito}}$ medio de ADAPT $\ell = 2$ es 2.0×10^{-2} , contra 1.1×10^{-5} de QAOA: 1750 veces más.
- **Contra QAOA INTERP, el resultado depende de la métrica.**
 - En la fracción con $p_{\text{éxito}} \geq 0.1$ (fig. 6a), INTERP gana en $n = 5-7$. Su $p_{\text{éxito}}$ es moderado pero nunca colapsa: en $n = 5$ el mínimo sobre las 20 instancias es 0.32, y en $n = 6$, 0.13.

Tabla 11: ADAPT ($k \leq 80$) contra QAOA ($p = 40$). “Tasa” es la fracción con $p_{\text{éxito}} \geq 0.1$: para ADAPT, promediada sobre instancias y órdenes; para QAOA, la fracción de instancias cuyo mejor reinicio resuelve. “Media” es $p_{\text{éxito}}$ promedio: sobre corridas para ADAPT, sobre todos los reinicios para QAOA y sobre instancias para INTERP. “Tope” es la fracción de corridas de ADAPT $\ell = 2$ que llegan a 80 parámetros.

n	azar	ADAPT $\ell = 1$		ADAPT $\ell = 2$			QAOA, reinicios		QAOA, INTERP	
		tasa	media	tasa	media	tope	tasa	media	tasa	media
5	2.5×10^{-2}	0.83	7.7×10^{-1}	0.94	9.4×10^{-1}	0.05	1.00	2.4×10^{-1}	1.00	5.6×10^{-1}
6	8.2×10^{-3}	0.67	4.4×10^{-1}	0.85	6.5×10^{-1}	0.16	0.10	3.9×10^{-2}	1.00	3.0×10^{-1}
7	2.7×10^{-3}	0.48	2.7×10^{-1}	0.56	4.0×10^{-1}	0.18	0.00	7.2×10^{-3}	0.80	1.2×10^{-1}
8	9.1×10^{-4}	0.25	1.3×10^{-1}	0.56	4.1×10^{-1}	0.61	0.00	1.6×10^{-3}	0.00	3.5×10^{-2}
9	3.0×10^{-4}	0.13	7.4×10^{-2}	0.20	8.8×10^{-2}	0.51	0.00	3.3×10^{-4}	0.00	1.1×10^{-2}
10	1.0×10^{-4}	0.07	2.6×10^{-2}	0.05	2.3×10^{-2}	0.67	0.00	1.2×10^{-4}	0.00	4.7×10^{-3}
11	3.4×10^{-5}	0.03	3.5×10^{-3}	0.07	1.8×10^{-2}	0.83	0.00	3.2×10^{-5}	0.00	1.5×10^{-3}
12	1.1×10^{-5}	0.00	1.4×10^{-10}	0.10	2.0×10^{-2}	0.84	0.00	1.1×10^{-5}	0.00	5.9×10^{-4}

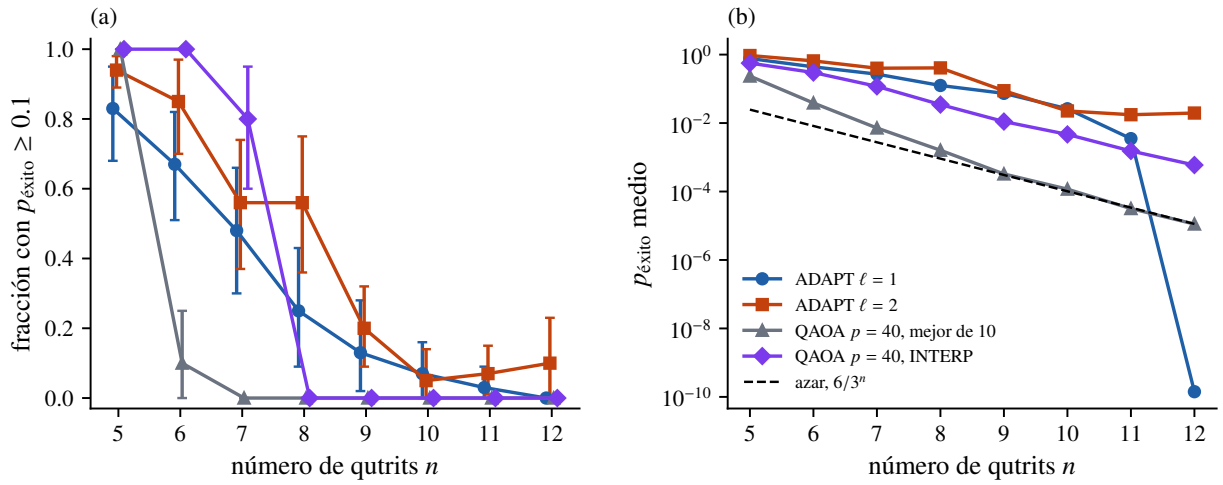


Figura 6: (a) Fracción con $p_{\text{éxito}} \geq 0.1$ contra n , con IC del 95 % por bootstrap sobre instancias. (b) $p_{\text{éxito}}$ medio contra n , con la probabilidad de acertar al azar.

- Con un umbral más exigente, $p_{\text{éxito}} \geq 0.5$, gana ADAPT $\ell = 2$ en todo n : 0.94 contra 0.65 en $n = 5$, 0.53 contra 0.00 en $n = 6$ y 0.28 contra 0.00 en $n = 7$.
- En $p_{\text{éxito}}$ medio (fig. 6b) gana ADAPT en todo n . Desde $n = 8$ INTERP ya no resuelve ninguna instancia, y ADAPT $\ell = 2$ sigue resolviendo más de la mitad en $n = 8$.
- ADAPT usa además menos parámetros: en $n \leq 7$ converge con una mediana de 10 a 42 parámetros, contra los 80 de QAOA.
- **QAOA con reinicios aleatorios sólo resuelve $n = 5$.** Desde $n = 9$ su $p_{\text{éxito}}$ medio queda al nivel del azar. Sólo entre el 50 y el 69 % de los reinicios converge dentro de 1000 iteraciones, según n .
- **El problema de QAOA es la optimización, no el circuito.** Con el mismo circuito, INTERP resuelve todas las instancias en $n = 5, 6$ y el 80 % en $n = 7$; en $n = 12$ su mediana es 5.9×10^{-4} , 52 veces el azar.
- **A igual número de parámetros** (fig. 7), ADAPT e INTERP coinciden en los primeros ~ 5 parámetros. Después ADAPT crece mucho más rápido: con unos 20 parámetros ya supera lo que QAOA INTERP alcanza con 80.
- **ADAPT es “todo o nada”.** Cada corrida termina resolviendo, con $p_{\text{éxito}}$ entre 0.5 y 1, o muy por debajo del azar. Por eso su tasa cae con n aunque su $p_{\text{éxito}}$ medio siga muy por encima de QAOA.

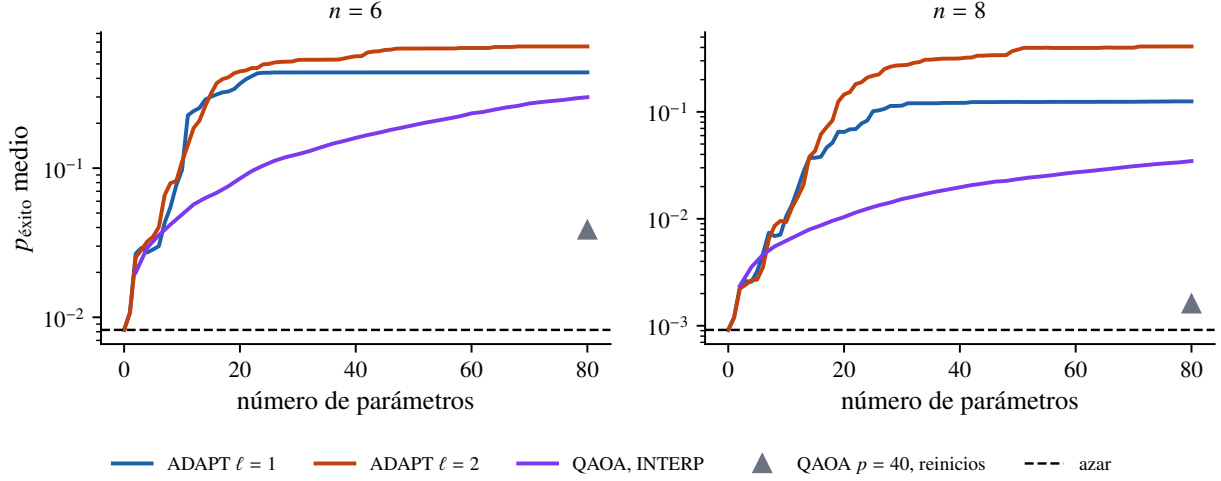


Figura 7: $p_{\text{éxito}}$ medio contra número de parámetros: ADAPT en cada iteración k (100 corridas por curva) y QAOA INTERP con $2p$ parámetros (20 instancias). El triángulo es el promedio de los 200 reinicios aleatorios con $p = 40$.

- En $n \geq 10$, $\ell = 2$ **queda limitado por el tope:** entre el 67 y el 84 % de sus corridas llega a 80 parámetros. En $n = 10$ no se separa de $\ell = 1$; en $n = 11$ y 12 sí, porque $\ell = 1$ converge antes a estados que casi nunca son la solución.

Conclusiones. Con el mismo presupuesto de 80 parámetros, ADAPT da probabilidades de éxito órdenes de magnitud mayores que QAOA con reinicios aleatorios. Ese protocolo, el mismo del paper para Max 3-Cut, no sirve para QAOA en MWNP: en un paisaje de 80 dimensiones casi nunca encuentra un buen mínimo. La referencia de QAOA para el paper debería ser INTERP, que es mucho más fuerte. Contra INTERP la comparación es más matizada: ADAPT concentra más probabilidad en la solución y escala mejor con n , pero en $n \leq 7$ INTERP logra un $p_{\text{éxito}}$ moderado en más instancias. Qué método “gana” depende de si interesa la probabilidad por disparo o la fracción de instancias con al menos una probabilidad baja de acierto.

4.5 Compuertas nativas

Contamos las compuertas de cada corrida igual que en el Apéndice B del paper: rotaciones de dos niveles $R^{(i,j)}$ (locales, de un qutrit) y compuertas Mølmer-Sørensen (no locales, de dos qutrits). Para ADAPT se toma el circuito en $k = \min(80, k_{\text{final}})$. Para QAOA el número de compuertas no depende de la instancia ni de los ángulos, sólo de n y p ; se cuenta en su forma más barata (capa de costo en la base de Gell-Mann), como en la Tabla IV del paper.

Tabla 12: Compuertas nativas: medianas sobre las 100 corridas de cada n para ADAPT, y el valor (único) de QAOA con $p = 40$.

n	ADAPT $\ell = 1$			ADAPT $\ell = 2$			QAOA $p = 40$		
	locales	MS	total	locales	MS	total	locales	MS	total
5	111	18	129	182	34	216	5400	1600	7000
6	225	36	261	350	78	429	7920	2400	10320
7	195	32	227	642	134	776	10920	3360	14280
8	279	46	325	1214	266	1488	14400	4480	18880
9	436	71	508	1198	261	1477	18360	5760	24120
10	470	77	546	1226	280	1514	22800	7200	30000
11	459	74	533	1226	288	1516	27720	8800	36520
12	612	100	712	1226	288	1513	33120	10560	43680

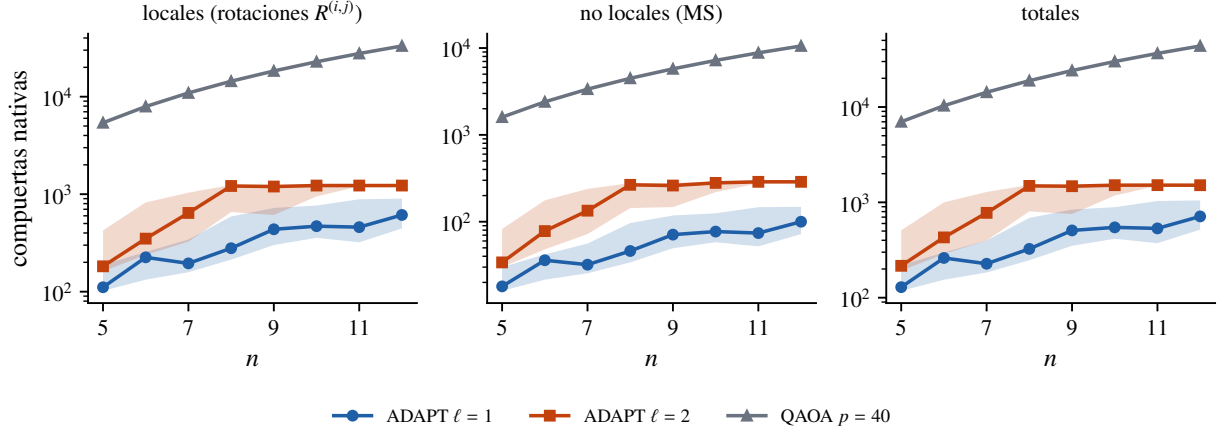


Figura 8: Compuertas nativas contra n : mediana (línea) y rango intercuartil (banda) de ADAPT, y QAOA con $p = 40$.

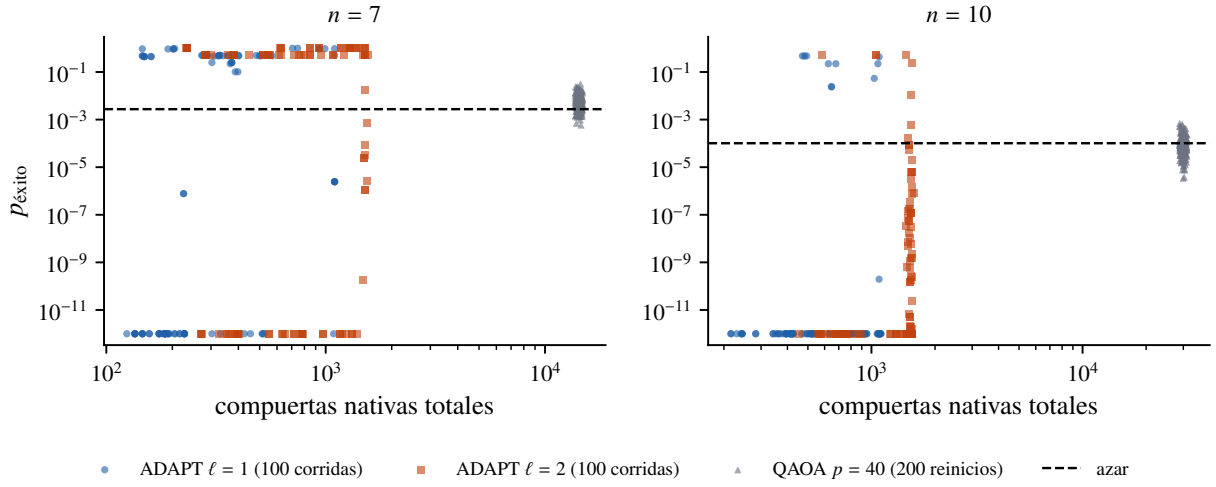


Figura 9: Cada corrida: compuertas totales contra $p_{\text{éxito}}$. Los reinicios de QAOA tienen todos el mismo número de compuertas; se desplazan levemente en x para verlos.

- ADAPT usa entre 13 y 60 veces menos compuertas que QAOA con 80 parámetros, tanto locales como MS (tabla 12). La razón es estructural: cada capa de QAOA aplica H_C completo, con un término por cada uno de los $\binom{n}{2}$ pares, mientras que cada parámetro de ADAPT es un solo operador de 1 a 4 qutrits.
- Las compuertas de ADAPT $\ell = 2$ se estancan en ~ 1500 desde $n = 8$ porque la mayoría de sus corridas llega al tope de 80 operadores (fig. 8).
- Con muchas menos compuertas, ADAPT alcanza $p_{\text{éxito}}$ que QAOA no alcanza (fig. 9). Las corridas de ADAPT que fallan, en cambio, quedan muy por debajo del azar (sección 4.8).

4.6 Comparación al estilo del paper

Las Figs. 1–3 del paper grafican, contra el número de parámetros, la mediana de QAOA sobre reinicios aleatorios con el rango intercuartil sombreado. Para hacer lo mismo corrimos QAOA con 10 reinicios por instancia en $p = 1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 25, 30, 35$ y 40, para $n = 5\text{--}10$ (fig. 10). En ADAPT la mediana y el rango se toman sobre las 100 corridas en cada iteración k . Como $E_0 = 0$, en lugar del error relativo se usa E/E_{unif} : la energía dividida por la de la superposición uniforme (el estado inicial). Vale 1 si el algoritmo no mejoró nada y tiende a 0 en la solución.

- **QAOA con reinicios aleatorios casi no baja la energía.** Su mediana de E/E_{unif} queda entre ~ 0.4 ($n = 6$) y ~ 1 ($n = 10$), y su $p_{\text{éxito}}$ se despega poco del azar. En $n = 8$ y 10 es incluso peor con más

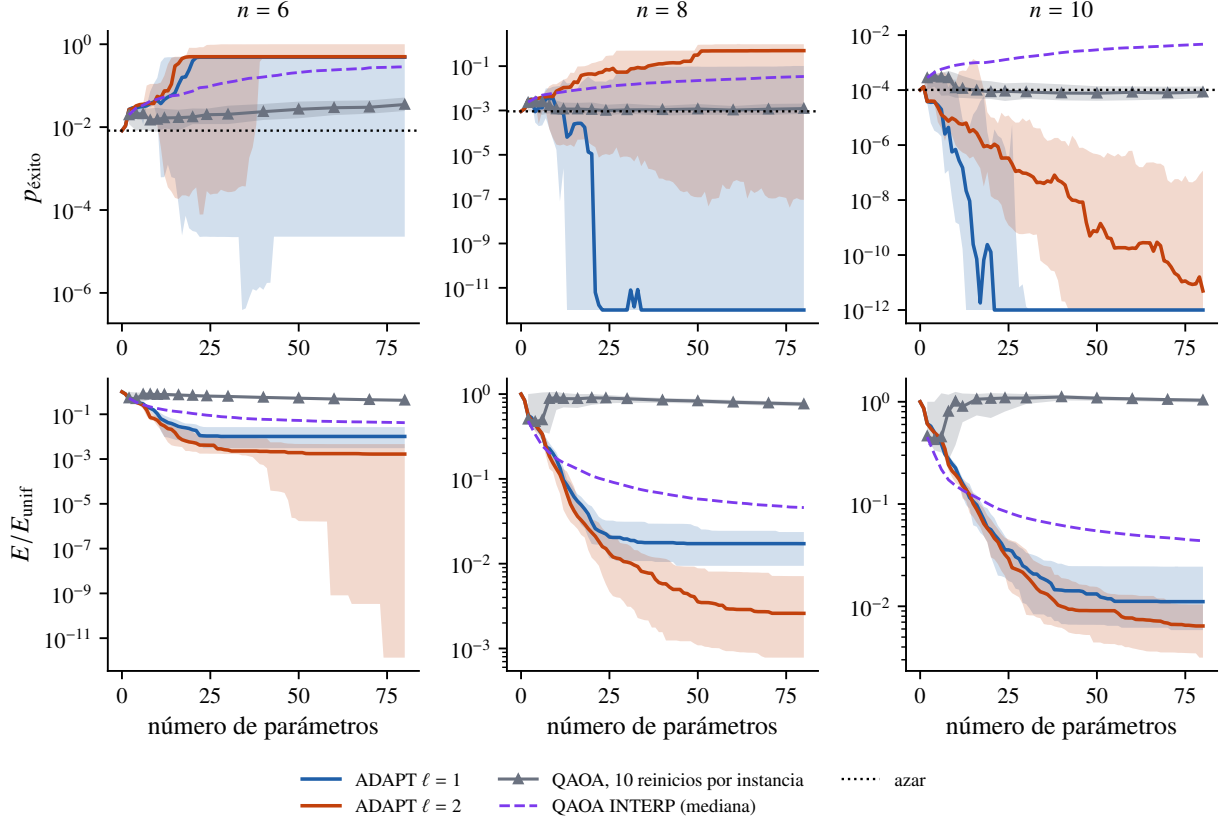


Figura 10: Mediana (línea) y rango intercuartil (banda) contra número de parámetros. Arriba, $p_{\text{éxito}}$; abajo, E/E_{unif} . ADAPT: 100 corridas por curva. QAOA con reinicios: 200 por punto (20 instancias $\times$ 10). La línea discontinua es la mediana de QAOA INTERP sobre las 20 instancias.

capas: baja a ~ 0.5 en $p = 1-2$ y vuelve a ~ 1 desde $p \approx 4$, porque con más parámetros el punto de partida aleatorio cae en un paisaje peor.

- **INTERP mejora de forma suave y consistente:** su energía baja con p y su $p_{\text{éxito}}$ queda siempre por encima del azar.
- **ADAPT baja la energía mucho más que ambos** ($E/E_{\text{unif}} \sim 10^{-2}$ o menos), pero su $p_{\text{éxito}}$ es bimodal: la mediana sigue a la mitad que falla. En $n = 10$ la mediana de $p_{\text{éxito}}$ de ADAPT queda bajo el azar, mientras la de INTERP queda sobre él, aunque en $p_{\text{éxito}}$ medio ADAPT gana (fig. 6b). La mediana describe la corrida típica y la media lo que se espera medir en promedio; con ADAPT las dos cuentan historias distintas.

4.7 Una métrica normalizada

Propuesta del grupo:

$$G = \frac{p_f - p_i}{p_M - p_i}, \quad p_i = \frac{6}{3^n}, \quad p_M = 1, \quad (26)$$

donde p_f es el $p_{\text{éxito}}$ final, p_i el de la superposición uniforme (el azar) y p_M el máximo posible. $G = 1$ es resolver con certeza, $G = 0$ es no mejorar al azar y $G < 0$ es empeorarlo. Como p_i es muy chico para $n \geq 7$, el valor medio de G es prácticamente el $p_{\text{éxito}}$ medio; lo que aporta G es el *signo*: cuántas corridas terminan peor que el azar (fig. 11).

- En G medio, ADAPT (con el mejor de sus dos pools) queda primero en todo n , e INTERP segundo.
- Pero la fracción de corridas *peor que el azar* crece con n para ADAPT: más del 80 % en $n \geq 10$. INTERP nunca empeora el azar, y los reinicios de QAOA quedan en medio.

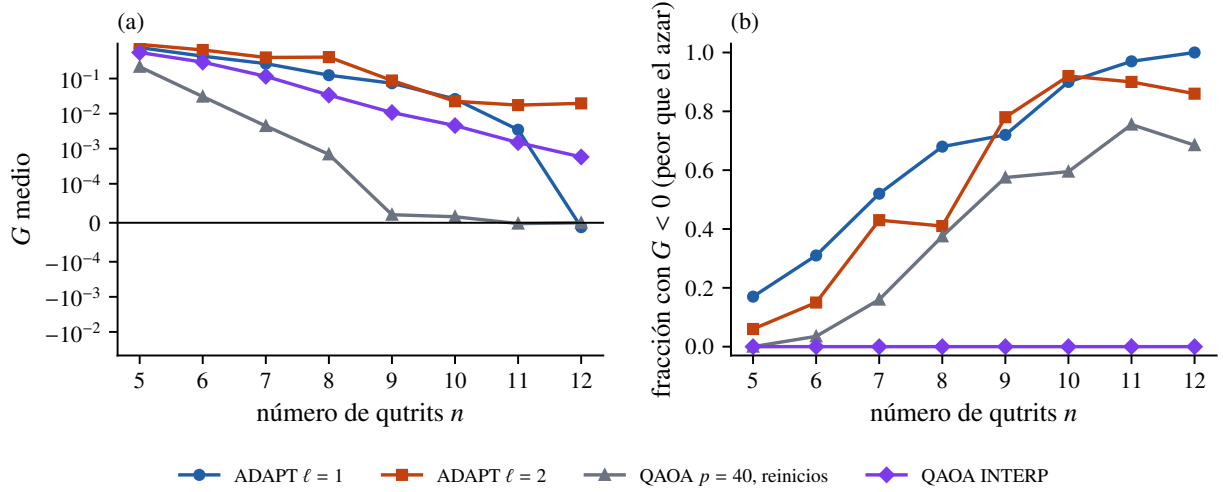


Figura 11: (a) G medio contra n , en escala simétrica logarítmica. (b) Fracción de corridas o reinicios con $G < 0$, es decir, peor que el azar.

- ADAPT es una estrategia de alto riesgo y alto retorno: concentra el estado; si acierta gana mucho, y si no, deja la solución con menos peso que el que tenía al partir.

4.8 ¿Por qué ADAPT puede terminar peor que el azar?

Llama la atención que en $n = 12$ el $p_{\text{éxito}}$ medio de ADAPT $\ell = 1$ quede bajo el azar (fig. 6b): el máximo sobre sus 100 corridas es 1.4×10^{-8} , contra 1.1×10^{-5} del azar. No es que ADAPT esté maximizando la energía. En esas mismas corridas:

- la energía baja a una mediana de $E/E_{\text{unif}} = 7.8 \times 10^{-3}$: *ADAPT* *minimiza*; las 10 cadenas más probables suman una mediana *el estado es muy concentrado*;
- esas cadenas son particiones *casi* buenas: la más probable tiene un desbalance (suma de la caja más llena menos la más vacía) típicamente entre 3 y 8, contra 0 de la solución;
- el 75 % de las corridas termina por el criterio de gradiente: es un mínimo local.

La superposición uniforme reparte la probabilidad en partes iguales, así que la solución tiene $6/3^n$. ADAPT concentra casi toda la probabilidad en unas pocas particiones de baja energía; si la solución no está entre ellas, su peso cae muy por debajo de $6/3^n$. Bajar la energía *media* no obliga a acercarse a la solución exacta, porque el fondo del espectro es denso en particiones casi balanceadas. Es el mismo mecanismo de las “respuestas perdidas” (fig. 16): en esa corrida la solución llegó a tener $p_{\text{éxito}} = 0.11$, y después el estado se concentró en otras particiones de energía aún menor en promedio, hasta dejarla en $\sim 10^{-22}$.

4.9 Varios órdenes como reinicios

Los 5 órdenes de cada instancia funcionan como 5 reinicios de ADAPT. Un experimento que no conoce la solución puede quedarse con el de menor energía final, igual que se hace con los reinicios de QAOA. La tabla 13 compara la tasa por orden con ese “mejor de 5”.

- Elegir por energía recupera casi todos los éxitos posibles: “mejor de 5” queda muy cerca de “alguno”. Cuando una corrida resuelve, en general también es la de menor energía.
- En $n \geq 9$ la tasa casi se duplica: el éxito de ADAPT en n grande depende sobre todo de acertar el desempate correcto, y probar varios órdenes es una forma barata de explorarlo.
- Con 5 corridas de ADAPT se resuelven más instancias que con 10 reinicios de QAOA en todo $n \geq 6$ (en $n = 5$ empatan con $\ell = 2$), y con muchas menos compuertas por corrida (sección 4.5).

Tabla 13: Fracción de instancias resueltas ($p_{\text{éxito}} \geq 0.1$). “Tasa”: promedio sobre órdenes. “Mejor de 5”: el orden de menor energía final, sin usar la solución. “Alguno”: resuelve en al menos un orden (cota superior). Para QAOA, el reinicio de menor energía de 10.

n	ADAPT $\ell = 1$			ADAPT $\ell = 2$			QAOA $p = 40$
	tasa	mejor de 5	alguno	tasa	mejor de 5	alguno	mejor de 10
5	0.83	0.90	0.90	0.94	1.00	1.00	1.00
6	0.67	0.85	0.85	0.85	0.90	0.90	0.10
7	0.48	0.65	0.70	0.56	0.70	0.75	0.00
8	0.25	0.35	0.35	0.56	0.65	0.65	0.00
9	0.13	0.25	0.25	0.20	0.40	0.45	0.00
10	0.07	0.15	0.15	0.05	0.10	0.10	0.00
11	0.03	0.05	0.05	0.07	0.15	0.20	0.00
12	0.00	0.00	0.00	0.10	0.15	0.15	0.00

4.10 Varianza del gradiente: mapa en n y k

Un barren plateau es una varianza del gradiente que cae exponencialmente con el número de qudits, para circuitos de profundidad suficiente. Para un circuito que se acerca a un 2-diseño en qutrits, cae como 3^{-n} . Medimos esa varianza en función de n y del número de operadores (capas) k :

- $n = 5-12$, con las 20 instancias del benchmark por tamaño;
- $k = 1, 2, 4, 8, 16, 32, 64$;
- dos familias de circuitos: (i) k operadores sorteados al azar del pool ($\ell = 1$ y $\ell = 2$), la pregunta de barren plateau para el pool; (ii) los primeros k operadores que eligió ADAPT en la corrida real de cada instancia, el circuito que de verdad se usa;
- 100 puntos de parámetros uniformes en $[-\pi, \pi]^k$ por circuito, gradiente exacto, y H_C normalizado por su rango espectral para que la escala no crezca con n .

Para cada circuito se promedia $\text{Var}[\partial_{\theta_j} E]$ sobre los parámetros j , y se reporta la mediana sobre instancias (figs. 12 y 13).

- **La varianza decrece con n en todas las familias, pero más lento que un 2-diseño.** Para $k = 16$, un ajuste $e^{-\alpha n}$ da $\alpha = 0.41$ ($\ell = 1$) y $\alpha = 0.56$ ($\ell = 2$), contra $\ln 3 \approx 1.10$. Con $k = 64$ y $\ell = 2$ la caída se acerca a 3^{-n} hasta $n \approx 9$ y luego se aplanan.
- **Los circuitos que elige ADAPT tienen más gradiente que los aleatorios:** para $k = 16$, la mediana es 3 veces ($\ell = 1$) y 19 veces ($\ell = 2$) la de circuitos sorteados del mismo pool, y decrece con $\alpha = 0.34$ y 0.29.
- **El pool $\ell = 2$ es más propenso a aplanarse con profundidad:** sus circuitos sorteados con $k = 64$ tienen la menor varianza del mapa en $n = 12$. Es lo esperable: sus operadores de 3 y 4 qutrits hacen al circuito más expresivo.
- **Límites:** $k \leq 64$ puede no alcanzar la profundidad de un 2-diseño en $n = 12$, y con $n \leq 12$ no se distingue un decaimiento exponencial lento de uno polinomial. Los puntos aleatorios no son los que recorre la optimización real, que parte de $\theta = 0$ (warm start).

4.11 Sensibilidad al orden, a la máquina y al rango

Resultados.

- **Orden.** En $n = 6$, con 10 órdenes por instancia y $\ell = 1$, 9 instancias se resuelven siempre, 3 nunca y 8 dependen del orden (fig. 15). En el experimento principal, entre 3 y 8 instancias por (n, ℓ) dependen del orden (tabla 8).

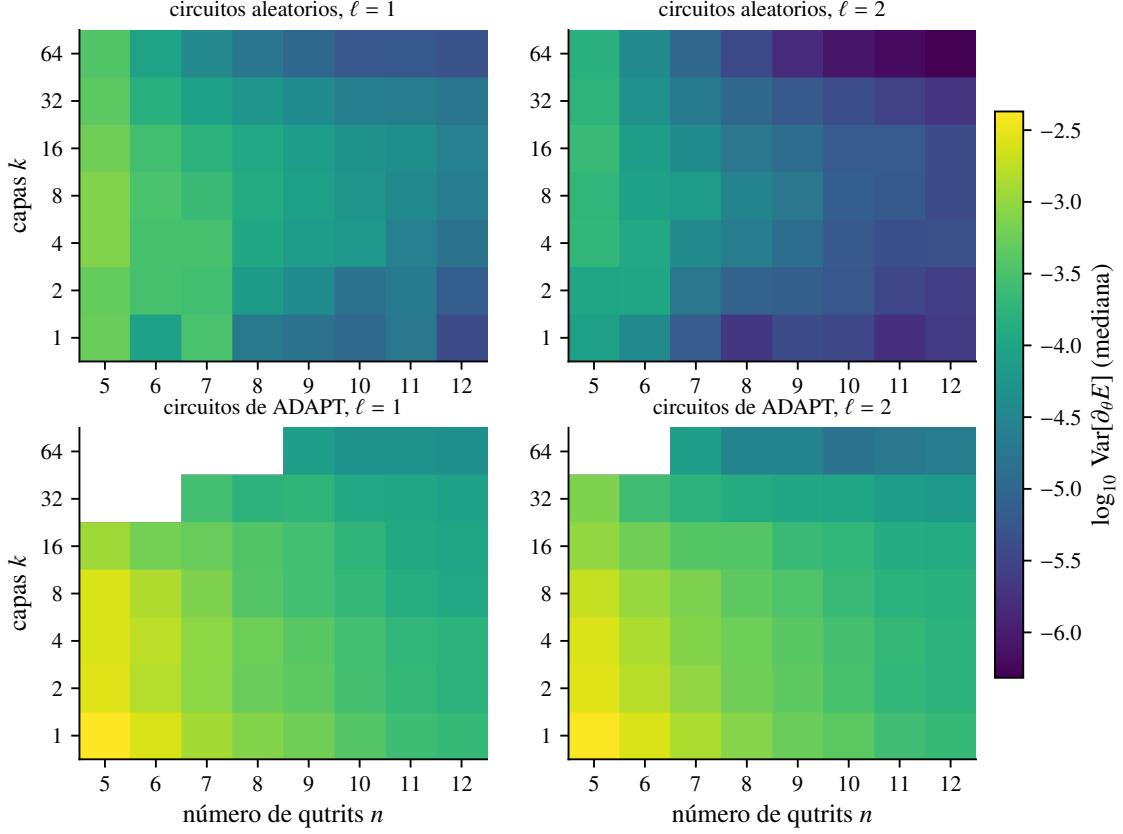


Figura 12: $\log_{10}$ de la mediana de $\text{Var}[\partial_\theta E]$ en función de n y k . Arriba, circuitos sorteados del pool; abajo, los circuitos que eligió ADAPT (faltan las celdas con k mayor que el largo de la corrida).

- El orden ascendente no es sistemáticamente peor: difiere de los otros órdenes en ≤ 0.08 en todos los casos salvo $n = 6$, $\ell = 1$ (0.50 contra 0.71).
- **Máquina.** Entre 1 y 2 hilos de BLAS en la misma máquina cambia el veredicto en 0/20 corridas; entre la laptop y BitWit, en 1/20. Las corridas en orden ascendente reproducen el veredicto de las corridas anteriores en 120/120 casos.
- **Rango.** En $n = 6$, las 9 instancias posibles en $[1, 8]$ no son más fáciles que las de $[1, 18]$: con $\ell = 1$ se resuelven 3/9 contra 10/20, y con $\ell = 2$, 8/9 contra 18/20.

Conclusiones. El criterio $\arg \max$ de ADAPT es sensible a empates y casi-empates. Una corrida individual mide tanto el algoritmo como un desempate arbitrario, y con distintas máquinas incluso el redondeo de BLAS puede decidir el empate. Toda tasa y toda comparación deben promediarse sobre órdenes, o sobre un desempate aleatorio con semilla, y compararse dentro de una misma máquina.

4.12 Otras observaciones

- **Respuestas perdidas.** Entre las corridas warm que fallan, 17/263 ($\ell = 1$) y 25/183 ($\ell = 2$) superaron $p_{\text{éxito}} = 0.1$ en alguna iteración y la perdieron mientras la energía seguía bajando (fig. 16). Minimizar $\langle H_C \rangle$ no obliga a concentrar el estado en la solución: el fondo del espectro es denso, y bajar la energía se logra repartiendo peso entre configuraciones casi óptimas.
- $p_{\text{éxito}} \approx 0.5$. 116 de 500 corridas con $\ell = 2$ terminan con el peso repartido por mitades entre la solución y una partición que intercambia dos números que difieren en 1. Parece un punto estacionario por simetría; queda por analizar.

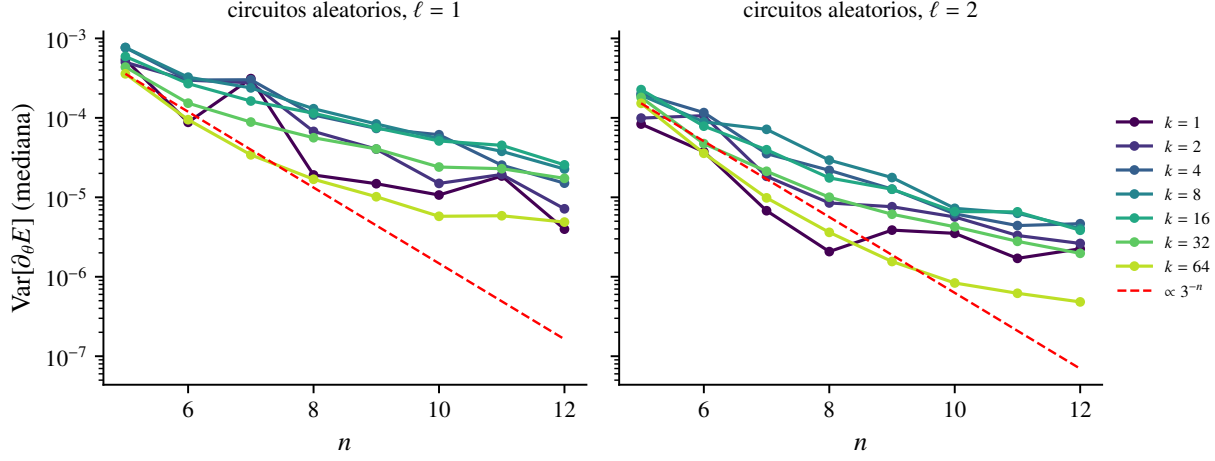


Figura 13: Cortes del mapa para circuitos sorteados: varianza contra n para cada k , con la referencia $\propto 3^{-n}$ anclada en $n = 5, k = 64$.

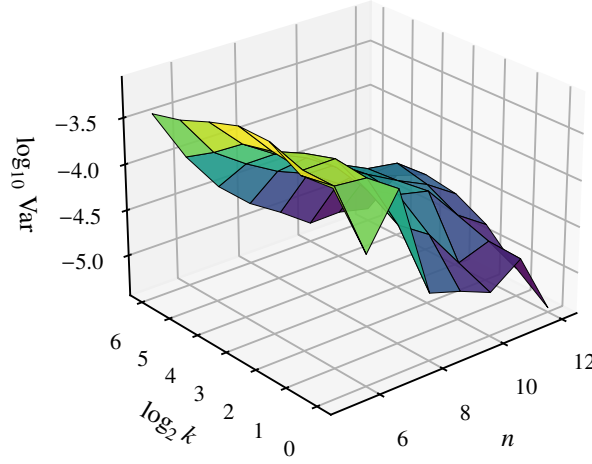


Figura 14: La misma información como superficie: $\log_{10} \text{Var}$ contra n y $\log_2 k$, circuitos sorteados del pool $\ell = 1$.

- **Varianza del gradiente:** ver la sección 4.10, que reemplaza la medición anterior con 3 instancias aleatorias.
- $n = 13, \ell = 1$: una corrida convergió en 64 parámetros (7.7 h). $p_{\text{éxito}}$ alcanzó 2.9×10^{-3} en $k = 12$ y terminó en 3.1×10^{-45} .

5 Conclusiones

1. MWNP con $k = 3$ es Max 3-Cut ponderado sobre K_n (ec. 14), y entra en el paper con el mismo pool, mixer y estado de referencia. Como $E_0 = 0$, la métrica tiene que ser $p_{\text{éxito}}$.
2. El pool $\ell = 2$ es significativamente mejor que $\ell = 1$, gracias a sus operadores de tres y cuatro sitios, a cambio de un pool de 30 a 80 veces más grande. Ambos pierden eficacia con n .
3. El warm start no es una desventaja. Con $\ell = 2$ es claramente mejor que reiniciar desde $\theta = 0$, tanto en la elección de operadores (ADAPT desde cero se detiene antes) como en la optimización de un circuito fijo. Con $\ell = 1$ las estrategias son equivalentes.

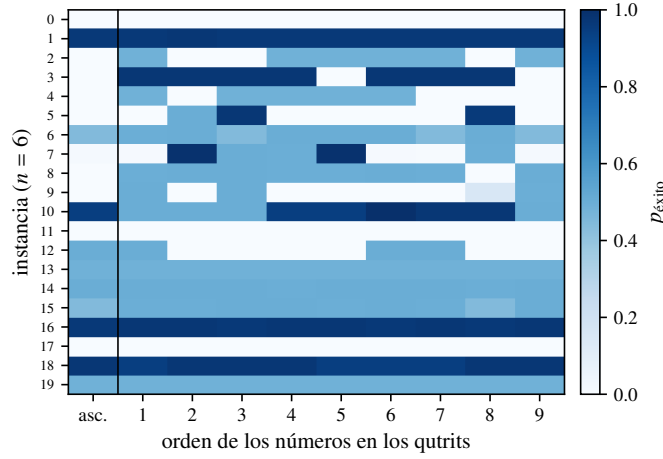


Figura 15: $p_{\text{éxito}}$ de las 20 instancias de $n = 6$ ($\ell = 1$, warm) bajo 10 órdenes. La primera columna es el orden ascendente.

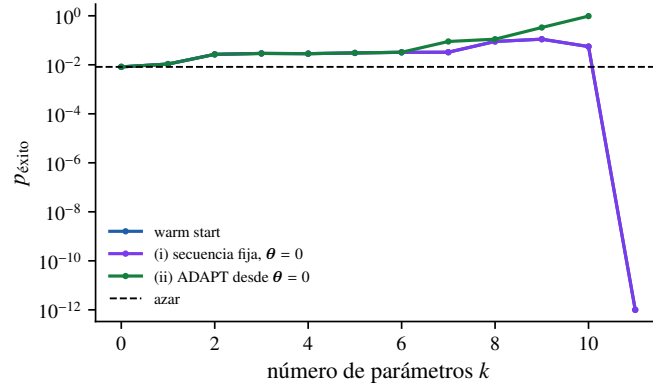


Figura 16: $p_{\text{éxito}}$ contra número de parámetros, $n = 6$, una instancia (orden ascendente), $\ell = 1$, con las tres estrategias. Warm llega a 0.11 y la pierde mientras la energía sigue bajando (ver la sección 4.8); ADAPT desde $\theta = 0$ termina en 0.97. Los valores menores que 10^{-12} se dibujan en ese piso. Es un ejemplo de un solo orden, no un promedio.

4. ADAPT supera ampliamente a QAOA con reinicios aleatorios, que no sirven en MWNP. Contra QAOA INTERP, la referencia justa, ADAPT gana en $p_{\text{éxito}}$ medio y a $n \geq 8$, e INTERP gana en fracción resuelta (umbral 0.1) a $n \leq 7$.
5. Las corridas individuales dependen del orden de los números y, en casi-empates, de la máquina. Las tasas deben promediarse sobre órdenes y compararse dentro de una misma máquina.

6 Notas para el manuscrito

- **Tamaño del pool $\ell = 2$.** El código del paper (`utilidades_bp.build_cd_pool`) concatena las listas de O_1 y O_3 , y $O_1 \subset O_3$. El pool $\ell = 2$ reportado cuenta cada operador de $\ell = 1$ dos veces: para K_6 son 2658 operadores distintos, no 2724, y para G_2 (Tabla II) son 1550, no 1600. La dinámica no cambia, porque `argmax` elige siempre la primera copia. Sí cambian el tamaño reportado y las mediciones de gradiente: $16 \times 1550 = 24\,800$ en vez de 25 600 en las Tablas II y III.
- **Mezclador de QAOA en Max 3-Cut.** El texto y el código no dicen lo mismo:
 1. *Lo que dice el texto.* La ec. (14) del paper define el circuito de QAOA como $\prod_k e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |\phi_g\rangle$, con el H_M de la ec. (9), $H_M = -\sum_j [(L_z^{(j)})^2 + \sqrt{2}L_x^{(j)}]$, cuyo estado fundamental es $|\phi_g\rangle = |+_3\rangle^{\otimes n}$. Así, QAOA es una versión digitalizada de la evolución adiabática que parte de ese estado.

2. *Lo que hace el código.* `funciones/utilidades_QAOA.py` define dos mezcladores: `Hm_qaoa_jx` (líneas 83–96), $H_M = \sum_j L_x^{(j)}$, y `Hm_qaoa_x_custom` (líneas 99–110), $H_M = \sum_j [(L_z^{(j)})^2 + \sqrt{2}L_x^{(j)}]$, que es el de la ec. (9) salvo un signo global (irrelevante: sólo invierte β). `preparar_qaoa_para_grafo` (línea 113) usa por defecto `mixer="jx"`, y el lanzador `cluster/main_QAOA.py` (líneas 88–94) también.
3. *El estado inicial no cambia con el mezclador.* Las líneas 157–160 toman siempre el fundamental de $-\sum_j [(L_z^{(j)})^2 + \sqrt{2}L_x^{(j)}]$, es decir $|+3\rangle^{\otimes n}$. Con el mezclador $\sum_j L_x^{(j)}$, ese estado *no* es su estado fundamental (ni siquiera un autoestado), así que el circuito corrido ya no es la digitalización de una evolución adiabática. Sigue siendo un *ansatz* variacional válido, pero no el que describe la ec. (14).
4. *Los resultados publicados usan $\sum_j L_x$.* Los dos archivos que carga `cuadernillos/comparacion_QAOA.ipynb` para las Figs. 1–3 y la Tabla I, `resultados_qaoa_n6_p20_jx_r25_grafos_1_all.json` y `resultados_qaoa_regulares_n6_p20_jx_r25_grafos_1_4.json`, llevan `jx` en el nombre, y sus 80 registros tienen `"mixer": "jx"`. El Apéndice B (compilación de QAOA, Tabla IV) sí dice $H_M = \sum_j L_x^{(j)}$, que es lo que se corrió.

Hay dos formas de arreglarlo: corregir el texto de la ec. (14) para que diga $\sum_j L_x^{(j)}$, o recorrer el QAOA de Max 3-Cut con `--mixer custom` para que las figuras correspondan a la ec. (9). La segunda es barata (8 grafos de $n = 6$) y además responde si QAOA mejora con el mezclador “correcto”. El QAOA de MWNP de este informe usa $\sum_j L_x^{(j)}$ para ser consistente con lo publicado, así que le aplica lo mismo.

- En la sección III.B aparece tres veces “wamr-start”.

7 Pendientes

- **Mezclador de la ec. (9) en QAOA** (ver sección 6):
 - Max 3-Cut: recorrer el QAOA de las Figs. 1–3 del paper con `--mixer custom` en `cluster/main_QAOA.py` (8 grafos, $n = 6$; barato), o corregir el texto de la ec. (14).
 - MWNP: repetir el benchmark de QAOA ($p = 40$, reinicios e INTERP) con `--mezclador x` en `cluster/qaoa_benchmark_mwnp.py`; $n \leq 11$ toma unas 9 h en BitWit.
- $\ell = 3$ en $n = 5$ –9 (factibilidad en estudio: el pool crece ~ 13 veces respecto de $\ell = 2$ y llega a operadores de 6 qutrits).
- INTERP para $n = 10$ –12, en curso en BitWit.
- Subir el tope de $\ell = 2$ en $n \geq 8$, donde la mayoría de las corridas llega al máximo.
- Explicar el estancamiento en $p_{\text{éxito}} \approx 0.5$.
- Conteo de compuertas nativas de ADAPT contra QAOA a igual número de parámetros en MWNP, como en la Tabla IV del paper.